Parte 3: Implementación de aprendizaje no supervizado con el algoritmo K-Means

```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.cluster import KMeans
import warnings

warnings.filterwarnings('ignore')
sns.set_style('darkgrid')

# Cargando la tabla anteriormente generada después de la limpieza de datos
data = pd.read_csv('Final_DataFrame.csv', index_col=0)
data['WEEK'] = pd.to_datetime(data['WEEK']) #Cambiando el formato de la columna "WEEK" a formato de fecha
data.head()
```

[147] Python

	WEEK	ITEM_CODE	TOTAL_UNIT_SALES	TOTAL_VALUE_SALES	TOTAL_UNIT_AVG_WEEKLY_SALES	REGION	MANUFACTURER	BRAND	ITEM_DESCRIPTION
0	2022-08-28	7501058792808BP2	0.006	0.139	1.000	TOTAL AUTOS AREA 5	RECKITT	VANISH	VANISHOXIACTIONROSADOYPACK120GR+MMCRYSTALWHITE...
1	2022-08-28	7501058715883	0.487	116.519	2.916	TOTAL AUTOS AREA 5	RECKITT	VANISH	VANISH OXI ACTION GOLD QUITAMANCHAS BOLSA 1.8K...
2	2022-08-28	7702626213774	1.391	68.453	5.171	TOTAL AUTOS AREA 5	RECKITT	VANISH	VANISH OXI ACTION ROSA QUITAMANCHAS DOYPACK 24...

Vamos a eliminar las columnas 'ITEM_CODE' y 'CATEGORY', pues no aportan información relevante para el análisis ('ITEM_CODE' solo es el ID del producto y 'CATEGORY' solo tiene una clase para todos los datos: FABRIC TREATMENT and SANIT, como fue visto en el EDA previamente realizado)

```python
data.drop(columns=['ITEM_CODE','CATEGORY'], inplace=True)
data.head()
```

[148] Python

	WEEK	TOTAL_UNIT_SALES	TOTAL_VALUE_SALES	TOTAL_UNIT_AVG_WEEKLY_SALES	REGION	MANUFACTURER	BRAND	ITEM_DESCRIPTION	FORMAT	ATTR1
0	2022-08-28	0.006	0.139	1.000	TOTAL AUTOS AREA 5	RECKITT	VANISH	VANISHOXIACTIONROSADOYPACK120GR+MMCRYSTALWHITE...	POLVO	SAFE BLEACH
1	2022-08-28	0.487	116.519	2.916	TOTAL AUTOS AREA 5	RECKITT	VANISH	VANISH OXI ACTION GOLD QUITAMANCHAS BOLSA 1.8K...	POLVO	SAFE BLEACH
2	2022-08-28	1.391	68.453	5.171	TOTAL AUTOS AREA 5	RECKITT	VANISH	VANISH OXI ACTION ROSA QUITAMANCHAS DOYPACK 24...	POLVO	SAFE BLEACH
3	2022-08-28	0.022	1.481	1.833	TOTAL AUTOS AREA 5	RECKITT	VANISH	VANISH OXI ACTION GOLD QUITAMANCHA AHORRO DEL ...	POLVO	SAFE BLEACH
4	2022-08-28	2.037	182.839	5.375	TOTAL AUTOS AREA 5	RECKITT	VANISH	VANISH INTELLIGENCE POLVO BOTE 450 GR NAL 7501...	POLVO	SAFE BLEACH

Selección de las características para entrenar el modelo de K-Means

Variables numéricas

A continuación, se seleccionarán las características para entrenar al modelo de K-Means. Como este algoritmo funciona calculando las distancias euclidianas entre cada dato, es un requisito que las características sean numéricas. En este caso, contamos con 3 variables numéricas: 'TOTAL_UNIT_SALES', 'TOTAL_VALUE_SALES', 'TOTAL_UNIT_AVG_WEEKLY_SALES', las cuales solo están relacionadas con las ventas. Pero, además de estas variables, es posible introducir la variable de 'WEEK' numérica bajo las transformaciones adecuadas.

```python
date_in_ns = data['WEEK'].astype('int64') #Convirtiendo las fechas en enteros (representados en nanosegundos)
data['WEEK_HOURS'] = date_in_ns // 10**9 // (60*60) #Definiendo otra columna, con las fechas pero como números (en horas)
data.head()
```

[149] Python

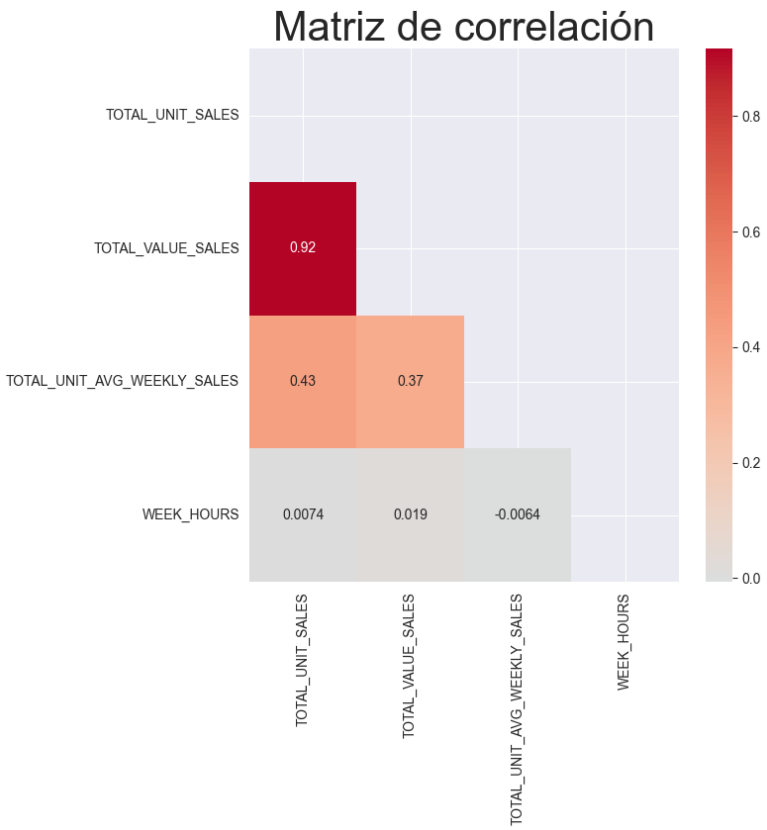
	WEEK	TOTAL_UNIT_SALES	TOTAL_VALUE_SALES	TOTAL_UNIT_AVG_WEEKLY_SALES	REGION	MANUFACTURER	BRAND	ITEM_DESCRIPTION	FORMAT	ATTR1
0	2022-08-28	0.006	0.139	1.000	TOTAL AUTOS AREA 5	RECKITT	VANISH	VANISHOXIACTIONROSADOYPACK120GR+MMCRYSTALWHITE...	POLVO	SAFE BLEACH
1	2022-08-28	0.487	116.519	2.916	TOTAL AUTOS AREA 5	RECKITT	VANISH	VANISH OXI ACTION GOLD QUITAMANCHAS BOLSA 1.8K...	POLVO	SAFE BLEACH
2	2022-08-28	1.391	68.453	5.171	TOTAL AUTOS AREA 5	RECKITT	VANISH	VANISH OXI ACTION ROSA QUITAMANCHAS DOYPACK 24...	POLVO	SAFE BLEACH
3	2022-	0.022	1.481	1.833	TOTAL AUTOS	RECKITT	VANISH	VANISH OXI ACTION GOLD QUITAMANCHA AHORRO DEL ...	POLVO	SAFE

Así, introducimos la nueva variable numérica 'WEEK_HOURS', la cual podremos utilizar para el modelo de KMeans.

Ahora bien, otro punto a verificar es si existe una fuerte correlación entre alguna de estas variables. Lo recomendable es que esto no exista, porque puede causar que el comportamiento de una variable tenga mayor peso que las otras. Esto lo podemos descubrir rápidamente si observamos la matriz de correlación.

```python
f, ax = plt.subplots(figsize = (7,7))

# Con esto quitamos los valores de la matriz identidad y los valores repetidos (nos ayudará a tener una mejor visualización)
mask = np.triu(np.ones_like(data.corr(), dtype=bool))

# Grafiquemos el heatmap
sns.heatmap(data.corr(), cmap='coolwarm', mask=mask, annot=True, center=0)

# Agrega un título a la gráfica
plt.title('Matriz de correlación', fontsize=30)
plt.show()
```

[150] Python



Podemos ver que 'TOTAL_UNIT_SALES' y 'TOTAL_VALUE_SALES' están muy fuertemente correlacionadas, lo cual tiene sentido porque una representa las ventas unitarias totales y la otra las ventas monetarias totales (conceptos que se relacionan con una simple multiplicación), por lo tanto, se descartará una de estas para evitar la colinealidad de las características para el modelo (se descartará 'TOTAL_VALUE_SALES').

Variables categóricas

Ahora bien, aunque KMeans solo use características numéricas, esto no significa que no sea posible incluir las variables categóricas al entrenamiento del modelo, pues se puede realizar una transformación numérica adecuada de las mismas. Las variables de este tipo de interés son 'REGION' (que nos da información de la región geográfica de las ventas), 'FORMAT' y 'SEGMENT' (que nos da información de la categoría a la que pertenecen los porductos).

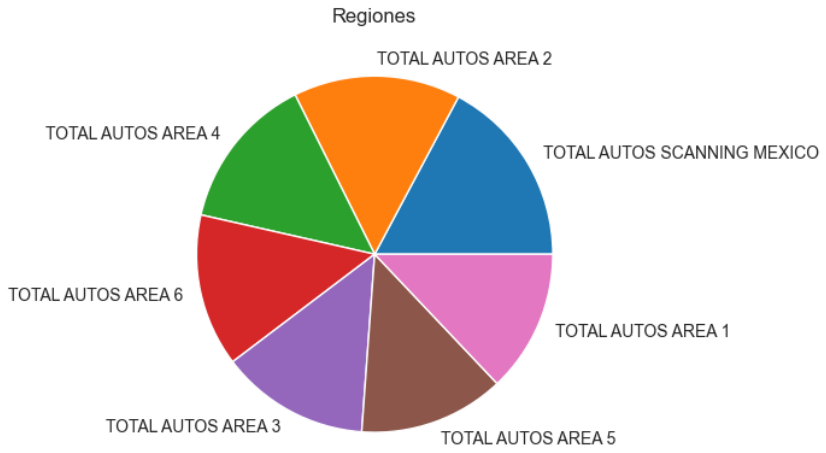
Dado que estas variables categóricas no son ordinales (no tienen algún tipo de jerarquía), darles etiquetas como "0, 1, 2" no es viable, porque estaríamos creando "distancias artificiales" que el modelo de K-Means podría interpretar erróneamente, por lo que la mejor solución es aplciar codificación "one-hot", en el que se crearán columnas individuales por cada clase de carácter binario, indicando con un "1" si pertenece a esa clase y "0" en caso de que no. Solo hay que tener precaución al utilizar este método, porque si existen muchas clases diferentes, esto introducirá una cantidad excesiva de columnas a los datos, aumentando fertemente su dimensionalidad. Esto es un problema apra K-Means, pues sucede lo que se le conoce como "La Maldición de la Dimensionalidad", que aparece porque las distancias euclidainas calculadas tienden a 0 en muy altas dimensiones, haciendo parecer que todos los puntos (datos) están a la misma distancia.

A continuación, vamos a ver qué clases y columnas valen la pena conservar de estas variables.

REGION

```python
plt.pie(data['REGION'].value_counts(), labels=data['REGION'].value_counts().index)
plt.title('Regiones')
plt.show()
```

[151] Python

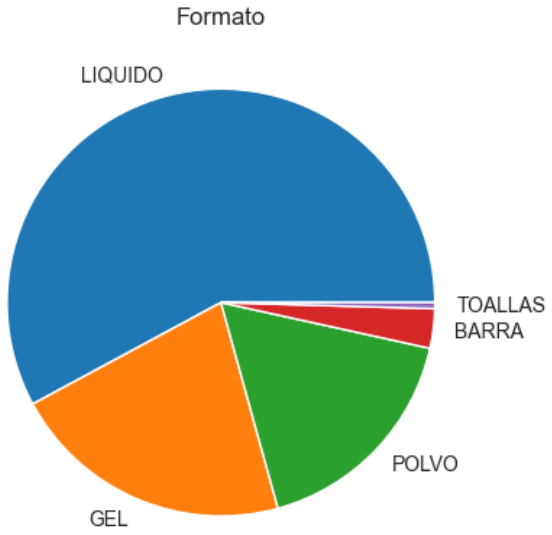


Observando esta gráfica de pastel, podemos ver que la columna 'REGION' cuenta con 7 clases (regiones) únicas, cada una estando balanceadas entre sí (es decir, que no hay unas pocas clases que dominen a las demás). Esto nos indica que las 7 regiones vale la pena incluirlas en el entrenamiento del modelo como características.

FORMAT

```python
plt.pie(data['FORMAT'].value_counts(), labels=data['FORMAT'].value_counts().index)
plt.title('Formato')
plt.show()
```

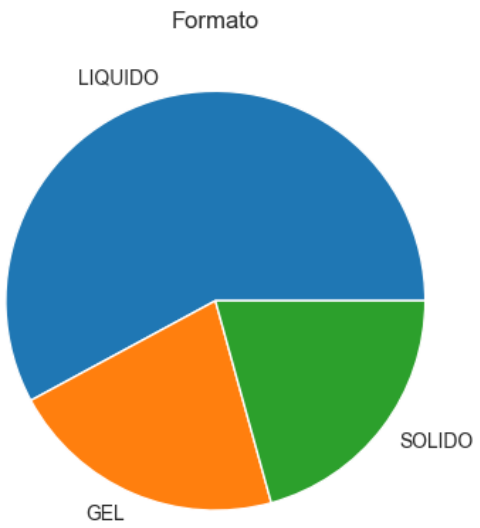
[152] Python



Para el caso de la columna 'FORMAT', tenemos 5 clases diferentes, siendo estas más desbalanceadas que las regiones. Dado que los formatos 'BARRA' y 'TOALLAS' representan una pequeña minoría de todos los datos presentes, se propone agrupar estos con la clase 'POLVO' en una nuva clase llamada 'SOLIDO', esto con el objetivo de reducir el número de clases y, por lo tanto, reduciendo la dimensionalidad de los datos después de aplicar codificación one-hot.

```python
def agrupar_solidos(formato): #Función para agrupar las clases 'POLVO','BARRA','TOALLAS' a una sola: 'SOLIDO'
    return 'SOLIDO' if formato in ['POLVO','BARRA','TOALLAS'] else formato

data.loc[:,'FORMAT'] = data['FORMAT'].apply(agrupar_solidos) # Aplicando la función a cada valor en la columna 'FORMAT'

# Verificando que se hayan agrupado correctamente
plt.pie(data['FORMAT'].value_counts(), labels=data['FORMAT'].value_counts().index)
plt.title('Formato')
plt.show()
```

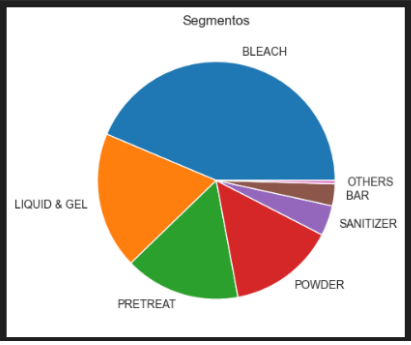
[153] Python



SEGMENT

```python
plt.pie(data['SEGMENT'].value_counts(), labels=data['SEGMENT'].value_counts().index)
plt.title('Segmentos')
plt.show()
```

[154] Python



En el caso de 'SEGMENT', tenemos 7 clases diferentes. Dado que tenemos información repetida de las categorías de los productos (Como "LIQUID & GEL" y "POWDER" que puede hacer referencia igual a las clases "LIQUIDO", "GEL" y "POLVO" de la columna 'FORMAT'), se decidió que no se tomará en cuenta esta columna, para evitar la introducción de más clases innecesarias al modelo

Otras variables categóricas

Nota: Otras variables como las que nos dan información de los atributos específicos de los productos ('ATTR1', 'ATTR2', 'ATTR3'), la marca ('BRAND') y la manufacturera ('MANUFACTURER') no serán consideradas por las siguientes razones:

Para las columans 'BRAND' y 'MANUFACTURER', la razón por la que no se incluyeron es porque hay muchas clases diferentes por cada una de estas variables, lo que aumentaría drásticamente las dimensiones de los datos de entrada para K-Means, lo que podría hacer que se agregue mucho ruido a los datos y que los resultados sean peores. Prueba de que existen muchas clases se puede ver a continuación:

```python
print('Marcas únicas de la columna BRAND:', data['BRAND'].unique(),'\n')
print('Manufactureros únicos de la columna MANUFACTURER:', data['MANUFACTURER'].unique())
```

[155] Python

```
Marcas únicas de la columna BRAND: ['VANISH' 'SHOUT' 'LA VALENCIANA' 'LOS PATITOS' 'OXI CLEAN' 'CLARASOL'
 'CLOROX' 'DR.BECKMANN' 'CLORALEX' 'CUELLOS PUNOS Y MANCHITAS' 'BANDERA'
 'BCLORHADA' 'PURO SOL' 'OTHERS MARCA UNIF.' 'TIDE' 'LYSOL' 'BLANCATEL'
 'CLORALUZ' 'CLORTEX' 'PATITO' 'SANYTOL' 'IBERIA' 'LA JAPONESA' 'HARPIC']

Manufactureros únicos de la columna MANUFACTURER: ['RECKITT' 'SC JOHNSON' 'BLANQUEADORA MEXICANA' 'CHURCH & DWIGHT'
 'CLARASOL' 'CLOROX' 'DELTA PRONATURA' 'INDS. ALEN' 'INDS. H24'
 'JABONERA LA ESPUMA' 'LA CORONA' 'OTHERS FABRICANTE UNIF.' 'P&G'
 'QUIMICA GONCAL' 'GRUPO AC MARCAS' 'IBERIA' 'LA JAPONESA']
```

Ahora bien, para los atributos, estos no se incluyeron dada la existencia de atributos repetidos por columna (como 'CLORO' y 'SANITIZANTE' que aparecen 2 veces en 2 columnas diferentes), además de la existencia de la clase 'NO DEFINIDO' en la columna 'ATTR3', la cual no nos da nada de información. Además, con las columnas 'FORMAT' y 'SEGMENT' ya tenemos una idea general del tipo de producto que se está manejando, por lo que agregar más información puede resultar en mayores complicaciones (por lo mismo de aumento de la dimensionalidad). Prueba de lo mencionado aquí se puede ver a continuación:

```python
print('Atributos únicos de la columna ATTR1:', data['ATTR1'].unique(),'\n')
print('Atributos únicos de la columna ATTR2:', data['ATTR2'].unique(),'\n')
print('Atributos únicos de la columna ATTR3:', data['ATTR3'].unique())
```

[156] Python

```
Atributos únicos de la columna ATTR1: ['SAFE BLEACH' 'PRELAVADOR' 'CLORO' 'SANITIZANTE']

Atributos únicos de la columna ATTR2: ['FABRIC TREATMENT' 'CLORO' 'SANITIZANTE']

Atributos únicos de la columna ATTR3: ['ROSA' 'BLANCO' 'PRE LAVADOR' 'NO DEFINIDO' 'MULTIUSOS' 'MASCOTAS'
 'SANITIZANTE' 'ROPA BEBE']
```

Selección final

Así, las características finales seleccionadas son las columnas nuéricas: 'TOTAL_UNIT_SALES', 'TOTAL_UNIT_AVG_WEEKLY_SALES' y 'WEEK_HOURS'; y las columnas categóricas: 'REGION', 'FORMAT'.

Como fue mencionado, será aplicado a las variables categóricas condificación "one-hot", lo cual se puede ahcer fácilmente con `pd.get_dummies`.

```python
X_num = data[['TOTAL_UNIT_SALES', 'TOTAL_UNIT_AVG_WEEKLY_SALES', 'WEEK_HOURS']]
X_num
```

[145] Python

	TOTAL_UNIT_SALES	TOTAL_UNIT_AVG_WEEKLY_SALES	WEEK_HOURS
0	0.006	1.000	461568
1	0.487	2.916	461568
2	1.391	5.171	461568
3	0.022	1.833	461568
4	2.037	5.375	461568
...	...	...	...
121997	1.192	4.000	468984
121998	1.947	2.797	468984
121999	18.013	9.102	468984
122000	21.152	22.172	468984
122001	1.468	2.068	468984

122002 rows × 3 columns

```python
X_categ = data[['REGION', 'FORMAT']]
X_categ = pd.get_dummies(X_categ)
X_categ
```

[157] Python

	REGION_TOTAL AUTOS AREA 1	REGION_TOTAL AUTOS AREA 2	REGION_TOTAL AUTOS AREA 3	REGION_TOTAL AUTOS AREA 4	REGION_TOTAL AUTOS AREA 5	REGION_TOTAL AUTOS AREA 6	REGION_TOTAL AUTOS SCANNING MEXICO	FORMAT_GEL	FORMAT_LIQUIDO	FORMAT_SOLIDO
0	0	0	0	0	1	0	0	0	0	1
1	0	0	0	0	1	0	0	0	0	1
2	0	0	0	0	1	0	0	0	0	1
3	0	0	0	0	1	0	0	0	0	1
4	0	0	0	0	1	0	0	0	0	1
...	...	...	...	...	...	...	...	...	...	...
121997	0	0	0	0	0	0	1	0	0	1
121998	0	0	0	0	0	0	1	0	1	0
121999	0	0	0	0	0	0	1	0	0	1
122000	0	0	0	0	0	0	1	0	0	1
122001	0	0	0	0	0	0	1	0	0	1

122002 rows × 10 columns

Así, tenemos un total de 3+10=13 columnas con las que será entrenado el modelo de K-Means. Aunque parecen ser datos con altas dimensiones, todavía es manejable. Aún así, es posible que esto genere ruido en los resultados o que la contribución de las variables categóricas domine el agrupamiento. Por eso, se ha decidido presentar 2 modelos:

- Un **modelo siple**, el cual solo sea entrenado con las 3 variables numéricas con las ventas totales y las fechas
- Un **modelo mixto**, el cual tenga las 3 variables numéricas + 10 variables "one-hot" categóricas

Reescalado de las características

Antes de proseguir, es necesario estandarizar las variables numéricas, para que las distancias que calcule el modelo K-Means no se vean afectadas por las diferentes escalas existentes.

```python
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_num_scaled = scaler.fit_transform(X_num)
X_num_scaled
```

[159] Python

```
array([[-0.22110293, -0.40176089, -0.24088462],
       [-0.18792124, -0.31716946, -0.24088462],
       [-0.12555899, -0.21761118, -0.24088462],
       ...,
       [ 1.02110618, -0.04405747,  1.66556282],
       [ 1.23764944,  0.53298321,  1.66556282],
       [-0.12024716, -0.35460867,  1.66556282]])
```

Modelado con K-Means

Modelo simple

Este modelo solo tomará en cuenta las 3 variables numéricas con la inforación de las ventas totales y las fechas.

```python
X = X_num_scaled
X
```

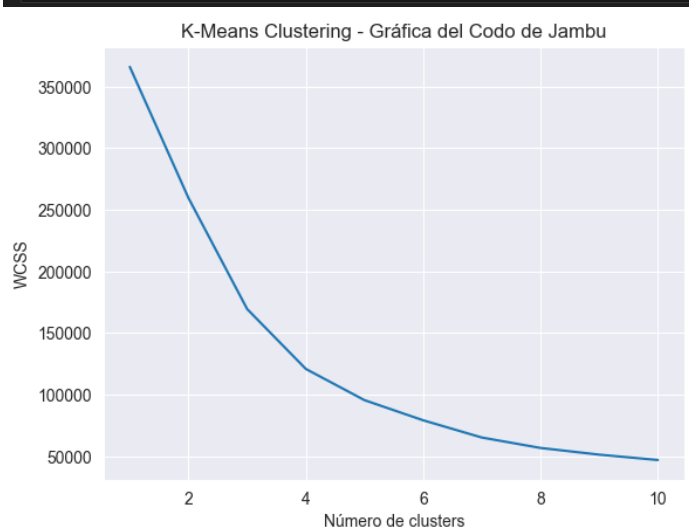
[168] Python

```
array([[-0.22110293, -0.40176089, -0.24088462],
       [-0.18792124, -0.31716946, -0.24088462],
       [-0.12555899, -0.21761118, -0.24088462],
       ...,
       [ 1.02110618, -0.04405747,  1.66556282],
       [ 1.23764944,  0.53298321,  1.66556282],
       [-0.12024716, -0.35460867,  1.66556282]])
```

Vamos a proabr diferentes modelos de K-Means con valores distintos de k, para ver cuál es el óptimo

```python
wcss = [] #Aquí se van a guardar las sumas de los errores cuadráticos de los catos a sus centroides
k_values = range(1,11)
for index in k_values: #probando múltiples valores para el núero de centroides en K-Means
    kmeans = KMeans(n_clusters=index, init='k-means++', random_state=42)
    kmeans.fit(X)

    wcss.append(kmeans.inertia_)

#Graficación del "Codo de Jambu"
plt.plot(k_values,wcss)
plt.title('K-Means Clustering - Gráfica del Codo de Jambu')
plt.xlabel('Número de clusters')
plt.ylabel('WCSS') #(Weighted Clusters Sum of Squares)
plt.show()
```

Python



Analizando la gráfica del Codo de Jambu, se pede ver que hay un codo muy marcado en k=4 o k=5, por lo que muy probablemente el valor óptimo de k sea ese. Aún así, es mejor comprobarlo utilizando una métrica numérica como el Criterio de la Silueata.

```python
# Criterio de Silueta ('Silhouette'), mientras más cercano sea a 1, será lo mejor
from sklearn.metrics import silhouette_score

# Dado que para calcular el criterio de la silueta se tienen lque calcular las distancias de cada par de datos existente y hay un total de 122002
# de datos, es uy complejo apra ser computado. En su lugar, se calculará en solo una muestra que sea 10 veces más pequeña (alrededor de ~10000)
sample_size = X.shape[0]//10

for j in range(3,11): # Se descarta k=1,2 porque tener solo 1 o 2 clusters no nos dará información relevante
    kmeans = KMeans(n_clusters=j, random_state=42)
    kmeans.fit_predict(X)
    # Calculamos el Score de Silueta
    score = silhouette_score(X, kmeans.labels_, metric='euclidean', sample_size=sample_size, random_state=42)
    print(f'Score Silhouette para k={j}: {score}')
```

Python

```
Score Silhouette para k=3: 0.5058351472882321
Score Silhouette para k=4: 0.5147104269641787
Score Silhouette para k=5: 0.523289420531097
Score Silhouette para k=6: 0.440999969039344
Score Silhouette para k=7: 0.4556967166374815
Score Silhouette para k=8: 0.46146160935271496
Score Silhouette para k=9: 0.4039383245680058
Score Silhouette para k=10: 0.4117797751099293
```

Descartando k=1 (dado que no se puede calcular esta métrica con k=1) y k=2 (dado que en la práctica tener solo 2 grupos para la segmentación de las ventas no es tan útil), podemos confirmar que k=5 es aquel que tiene el mejor valor en el Criterio de la Silueta (0.52), por lo que este es el valor más óptimo para nuestro modelo básico.

```python
# Rehaciendo el modelo de KMeans con el valor óptimo de k (k=3)
k_optim = 5
kmeans = KMeans(n_clusters=k_optim, init='k-means++', random_state=42)

cluster_values = kmeans.fit_predict(X) #Generamos un arreglo con las etiquetas a las que corresponde cada cluster
clusters = pd.DataFrame(cluster_values, columns=['CLUSTER_MODELO_BASICO']) #Creamos un DataFrame con las etiquetas anteriores
data_grouped = pd.concat([data,clusters], axis=1) #Anexamos esta columna al dataframe original
data_grouped
```

Python

LES	REGION	MANUFACTURER	BRAND	ITEM_DESCRIPTION	FORMAT	ATTR1	ATTR2	ATTR3	SEGMENT	WEEK_HOURS	CLUSTER_MODELO_BASICO
000	TOTAL AUTOS AREA 5	RECKITT	VANISH	VANISHOXIACTIONROSADOYPACK120GR+MMCRYSTALWHITE...	SOLIDO	SAFE BLEACH	FABRIC TREATMENT	ROSA	POWDER	461568	0
916	TOTAL AUTOS AREA 5	RECKITT	VANISH	VANISH OXI ACTION GOLD QUITAMANCHAS BOLSA 1.8K...	SOLIDO	SAFE BLEACH	FABRIC TREATMENT	ROSA	POWDER	461568	0
171	TOTAL AUTOS AREA 5	RECKITT	VANISH	VANISH OXI ACTION ROSA QUITAMANCHAS DOYPACK 24...	SOLIDO	SAFE BLEACH	FABRIC TREATMENT	ROSA	POWDER	461568	0
333	TOTAL AUTOS AREA 5	RECKITT	VANISH	VANISH OXI ACTION GOLD QUITAMANCHA AHORRO DEL ...	SOLIDO	SAFE BLEACH	FABRIC TREATMENT	ROSA	POWDER	461568	0
375	TOTAL AUTOS AREA 5	RECKITT	VANISH	VANISH INTELLIGENCE POLVO BOTE 450 GR NAL 7501...	SOLIDO	SAFE BLEACH	FABRIC TREATMENT	ROSA	POWDER	461568	0

Modelo mixto

Este modelo tomará en cuenta las 3 variables numéricas + 10 variables "one-hot" categóricas. Se espera que este modelo pueda detectar patrones más complejos que sean una mezcla de las ventas totales, fechas, las regiones y las categorías de los productos

```python
X = np.concatenate([X_num_scaled, X_categ.values], axis=1)
X
```

Python

```
array([[-0.22110293, -0.40176089, -0.24088462, ...,  0.        ,
        0.        ,  1.        ],
       [-0.18792124, -0.31716946, -0.24088462, ...,  0.        ,
        0.        ,  1.        ],
       [-0.12555899, -0.21761118, -0.24088462, ...,  0.        ,
        0.        ,  1.        ],
       ...,
       [ 1.02110618, -0.04405747,  1.66556282, ...,  0.        ,
        0.        ,  1.        ],
       [ 1.23764944,  0.53298321,  1.66556282, ...,  0.        ,
        0.        ,  1.        ],
       [-0.12024716, -0.35460867,  1.66556282, ...,  0.        ,
        0.        ,  1.        ]])
```

Ahora se realizarán varias pruebas con diferentes valores de k. En este caso, dada la mayor información que se tiene de los datos, se probará un rango más alto de clusters (de k=1 hasta k=40)

Modelo mixto

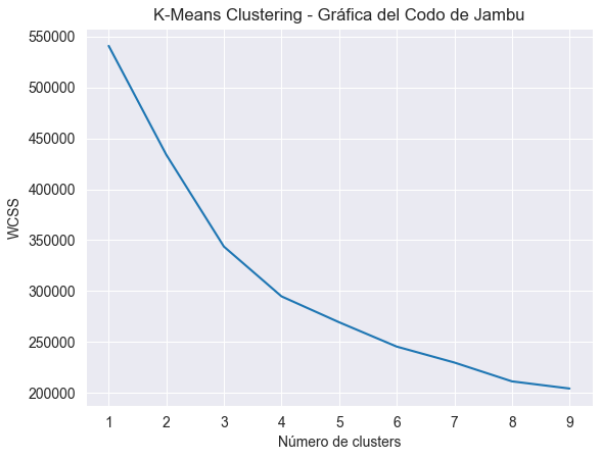
Este modelo tomará en cuenta las 3 variables numéricas + 10 variables "one-hot" categóricas. Se espera que este modelo pueda detectar patrones más complejos que sean una mezcla de las ventas totales, fechas, las regiones y las categorías de los productos

```python
X = np.concatenate([X_num_scaled, X_categ.values], axis=1)
X
```

[170] Python

```
array([[-0.22110293, -0.40176089, -0.24088462, ...,  0.        ,
         0.        ,  1.        ],
       [-0.18792124, -0.31716946, -0.24088462, ...,  0.        ,
         0.        ,  1.        ],
       [-0.12555899, -0.21761118, -0.24088462, ...,  0.        ,
         0.        ,  1.        ],
       ...,
       [ 1.02110618, -0.04405747,  1.66556282, ...,  0.        ,
         0.        ,  1.        ],
       [ 1.23764944,  0.53298321,  1.66556282, ...,  0.        ,
         0.        ,  1.        ],
       [-0.12024716, -0.35460867,  1.66556282, ...,  0.        ,
         0.        ,  1.        ]])
```

Ahora se realizarán varias pruebas con diferentes valores de k. En este caso, dada la mayor información que se tiene de los datos, se probará un rango más alto de clusters (de k=1 hasta k=40)



Analizando la gráfica, parece ser que alrededor de k=4 o k=6 está el codo. Aún así, esto no parece ser muy claro, por lo que se calcularán los criterios de la silueta para cada valor de k.

```python
# Criterio de Silueta ('Silhouette'), mientras más cercano sea a 1, será lo mejor
from sklearn.metrics import silhouette_score

sample_size = X.shape[0]//10

for j in range(3,10): # Se descarta k=1,2 porque tener solo 1 o 2 clusters no nos dará información relevante
    kmeans = KMeans(n_clusters=j, random_state=42)
    kmeans.fit_predict(X)
    # Calculamos el Score de Silueta
    score = silhouette_score(X, kmeans.labels_, metric='euclidean', sample_size=sample_size, random_state=42)
    print(f'Score Silhouette para k={j}: {score}')
```

[164] Python

```
Score Silhouette para k=3: 0.23326880239907996
Score Silhouette para k=4: 0.2379935105088761
Score Silhouette para k=5: 0.24038131822867295
Score Silhouette para k=6: 0.19826878183143787
Score Silhouette para k=7: 0.1872234050818655
Score Silhouette para k=8: 0.21300671226649792
Score Silhouette para k=9: 0.17921007857746812
```

En este caso, pasa algo interesante con los Criterios de Silueta: aunque para valores de k pequeños, k=5 parece ser el valor óptimo (pues se alcanza un máximo local de 0.240), para k >= 11 esta métrica aumenta lentamente, llegando a un máximo absoluto en k=38 con 0.392. Aunque parezca que k=38 sea la mejor elección a primera vista, el problema es que este alto número de clusters es muy probable que el modelo está siendo dominado por las etiquetas de las categorías, lo cual no es útil en la práctica porque no está encontrando patrones ocultos en las ventas. Esto se confirma viendo la gráfica del Codo de Jambu, en donde se puede ver que para k=11 en adelante, la curva se vuelve más lineal (hay menor ganancia de información por cluster para k >= 11).

Por cuestiones prácticas y, para tener un resultado más segmentado, se tomará k=8 en vez de k=5, porque este tiene un máximo local de 0.213 y queremos un número diferente de clusters al obtenido con solo las variables numéricas. Además, también se ve que en la gráfica del codo que el WCSS disminuye en menor medida después de k=8.

```python
# Rehaciendo el modelo de KMeans con el valor óptimo de k con baja resolución (k=7)
k_optim = 8

kmeans = KMeans(n_clusters=k_optim, init='k-means++', random_state=42)
cluster_values = kmeans.fit_predict(X) #Generamos un arreglo con las etiquetas a las que corresponde cada cluster
clusters = pd.DataFrame(cluster_values, columns=['CLUSTER_MODELO_MIXTO']) #Creamos un DataFrame con las etiquetas anteriores
df = pd.concat([data_grouped,clusters], axis=1) #Anexamos esta columna al dataframe original
df
```

[171] Python

TURER	BRAND	ITEM_DESCRIPTION	FORMAT	ATTR1	ATTR2	ATTR3	SEGMENT	WEEK_HOURS	CLUSTER_MODELO_BASICO	CLUSTER_MODELO_MIXTO
RECKITT	VANISH	VANISHOXIACTIONROSADOYPACK120GR+MMCRYSTALWHITE...	SOLIDO	SAFE BLEACH	FABRIC TREATMENT	ROSA	POWDER	461568	0	4
RECKITT	VANISH	VANISH OXI ACTION GOLD QUITAMANCHAS BOLSA 1.8K...	SOLIDO	SAFE BLEACH	FABRIC TREATMENT	ROSA	POWDER	461568	0	4
RECKITT	VANISH	VANISH OXI ACTION ROSA QUITAMANCHAS DOYPACK 24...	SOLIDO	SAFE BLEACH	FABRIC TREATMENT	ROSA	POWDER	461568	0	4
RECKITT	VANISH	VANISH OXI ACTION GOLD QUITAMANCHA AHORRO DEL ...	SOLIDO	SAFE BLEACH	FABRIC TREATMENT	ROSA	POWDER	461568	0	4
RECKITT	VANISH	VANISH INTELLIGENCE POLVO BOTE 450 GR NAL 7501...	SOLIDO	SAFE BLEACH	FABRIC TREATMENT	ROSA	POWDER	461568	0	4

Visualización de Clusters

Por simplicidad, se cambiará el nombre de las regiones, quitando el 'TOTAL AUTOS' en cada etiqueta

```python
df['REGION']=df['REGION'].replace(['TOTAL AUTOS AREA 1','TOTAL AUTOS AREA 2','TOTAL AUTOS AREA 3','TOTAL AUTOS AREA 4','TOTAL AUTOS AREA 5','TOTAL AUTOS AREA 6','TOTAL AUTOS SCANNIN
                                   ['AREA 1','AREA 2','AREA 3','AREA 4','AREA 5','AREA 6','SCANNING MEXICO'])
df['REGION'].unique()
```

Python

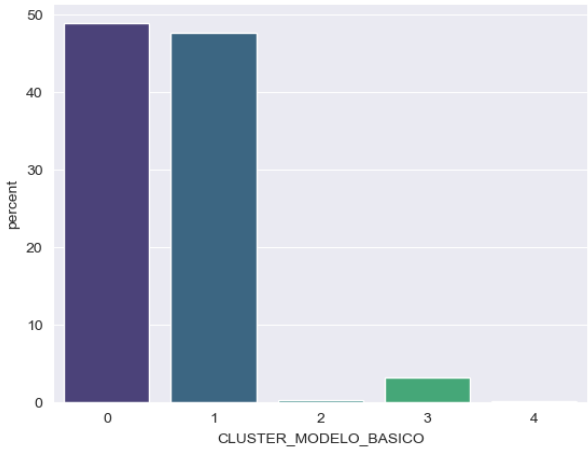
```
array(['AREA 5', 'AREA 6', 'SCANNING MEXICO', 'AREA 3', 'AREA 2',
       'AREA 1', 'AREA 4'], dtype=object)
```

Modelo simple

```python
sns.countplot(data=df, x='CLUSTER_MODELO_BASICO', palette='viridis', stat='percent')
plt.show()
```

0.8s

Python



```python
(df['CLUSTER_MODELO_BASICO'].value_counts() / len(df) * 100).sort_index()
```

[4] 0.0s Python

```
0    48.925427
1    47.641842
2     0.191800
3     3.152407
4     0.088523
Name: CLUSTER_MODELO_BASICO, dtype: float64
```

```python
# Creando un DataFrame para guardar el análisis de cada cluster
k=5
resumen_clusters_basico = pd.DataFrame(data=np.full((k,7),fill_value='-'),
                                       columns=['Representación del total','Regiones','Formato','Segmento','Ventas Unitarias Totales','Promedio de Ventas Unitarias','Fecha'],
                                       index=np.arange(k))
resumen_clusters_basico.index.name='Cluster'
resumen_clusters_basico.loc[:,'Representación del total']=['48.9%', '47.6%',  '0.2%', '3.2%',  '0.1%']
resumen_clusters_basico
```

[5] 0.0s Python

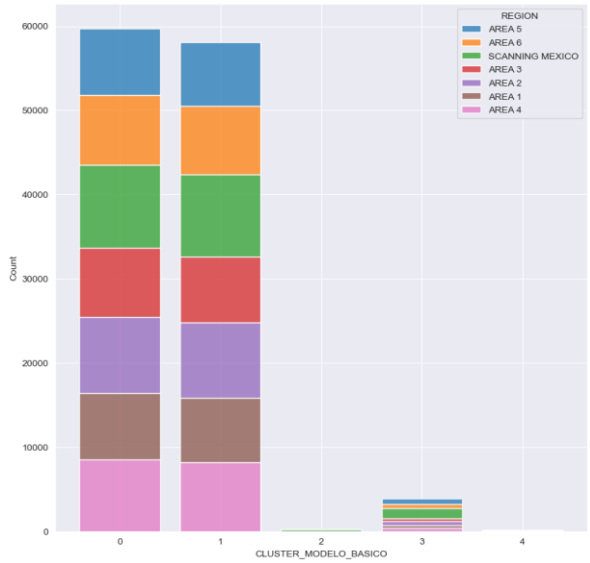
Cluster	Representación del total	Regiones	Formato	Segmento	Ventas Unitarias Totales	Promedio de Ventas Unitarias	Fecha
0	48.9%	-	-	-	-	-	-
1	47.6%	-	-	-	-	-	-
2	0.2%	-	-	-	-	-	-
3	3.2%	-	-	-	-	-	-
4	0.1%	-	-	-	-	-	-

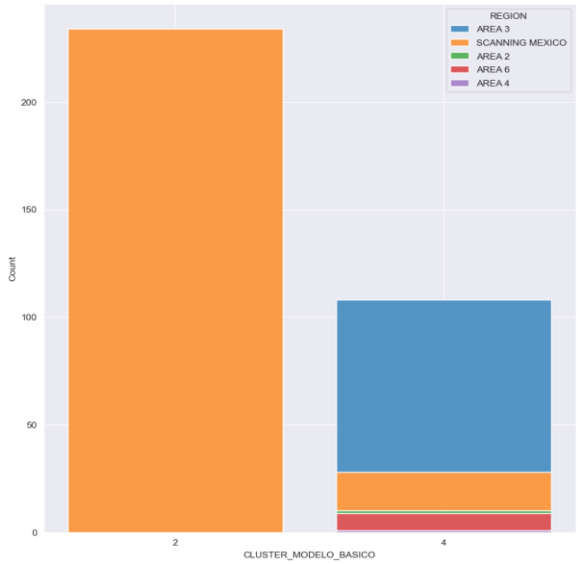
```python
def graficar_barras_apiladas(df, col_clusters, k, categoria, figsize): #Funcion para hacer gráficas de barras por cluster apiladas (divididas dada una categoría)
    f, ax = plt.subplots(figsize=figsize)

    df2 = df.copy()
    df2[col_clusters] = df2[col_clusters].astype('str') #Convirtiendo las etiquetas de los clusters en cadenas de texto
    clusters_labels=np.arange(k).astype(str)
    df2[col_clusters] = pd.Categorical(df2[col_clusters], clusters_labels) #Ordenando los clusters

    sns.histplot(data=df2, x=col_clusters, hue=categoria, multiple='stack', ax=ax, shrink=.8)

    return df2


def graficar_barras_minoria(df, col_clusters, clusters_minoria, categoria, figsize): #Misma funcion, pero ahora solo con los clusters que representan la minoría (especificados)
    f, ax = plt.subplots(figsize=figsize)

    df2=df.copy()
    df2=df2[df2[col_clusters].isin(clusters_minoria)]
    df2[col_clusters] = df2[col_clusters].astype('str') #Convirtiendo las etiquetas de los clusters en cadenas de texto
    clusters_labels = list(map(str,clusters_minoria))
    df2[col_clusters] = pd.Categorical(df2[col_clusters], clusters_labels) #Ordenando los clusters

    sns.histplot(data=df2, x=col_clusters, hue=categoria, multiple='stack', ax=ax, shrink=.8)
    plt.show()
```

[6] 0.0s Python

```python
df2 = graficar_barras_apiladas(df=df, col_clusters='CLUSTER_MODELO_BASICO', k=5, categoria='REGION', figsize=(10,10))

#Gráfica anterior pero solo con los clusters que representan la minoría
graficar_barras_minoria(df=df, col_clusters='CLUSTER_MODELO_BASICO', clusters_minoria=[2,4], categoria='REGION', figsize=(10,10))
```

[7] 1.3s Python





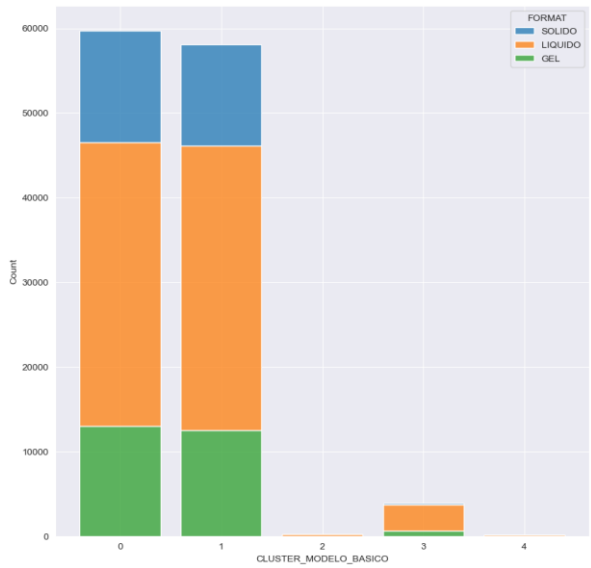
```python
resumen_clusters_basico.loc[2,'Regiones']='SCANNING MEXICO'
resumen_clusters_basico.loc[4,'Regiones']='AREA 3, SCANNING MEXICO'
resumen_clusters_basico.loc[[0,1,3],'Regiones'] = 'MIXTO'
resumen_clusters_basico
```

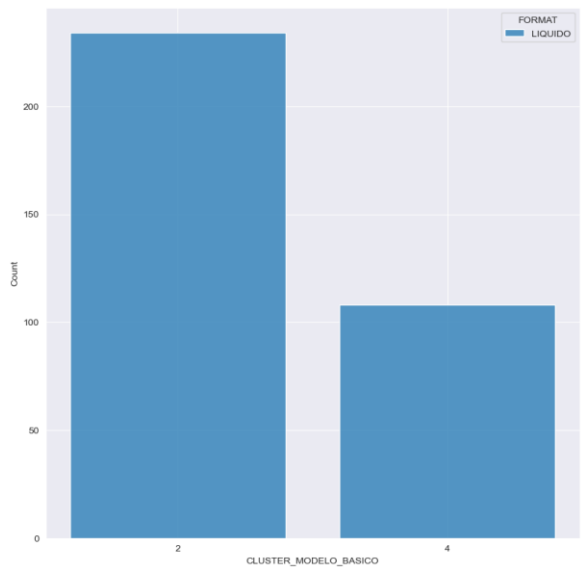
[8] ✓ 0.0s Python

	Representación del total	Regiones	Formato	Segmento	Ventas Unitarias Totales	Promedio de Ventas Unitarias	Fecha
Cluster							
0	48.9%	MIXTO	-	-	-	-	-
1	47.6%	MIXTO	-	-	-	-	-
2	0.2%	SCANNING MEXICO	-	-	-	-	-
3	3.2%	MIXTO	-	-	-	-	-
4	0.1%	AREA 3, SCANNING MEXICO	-	-	-	-	-

```python
df2 = graficar_barras_apiladas(df=df, col_clusters='CLUSTER_MODELO_BASICO', k=5, categoria='FORMAT', figsize=(10,10))

#Gráfica anterior pero solo con los clusters que representan la minoría
graficar_barras_minoria(df=df, col_clusters='CLUSTER_MODELO_BASICO', clusters_minoria=[2,4], categoria='FORMAT', figsize=(10,10))
```

[9] ✓ 1.1s Python





Aquí tenemos un problema: la variable 'FORMAT' está desbalanceada, dado que la clase "LIQUIDO" representa más de la mitad de todos los datos. Por lo tanto, si existe una mayoría de esta clase en algún cluster, no significa mucho, a menos que sea mayor a la proporción original. Para resolver esto, necesitas medir la **sobrerrepresentación**. Una forma de medirlo es con la siguiente métrica:

El Índice de Lift

Es la métrica más intuitiva. Compara la proporción de una clase dentro de un grupo frente a su proporción en toda la población. Si, en este caso, A representa uno de los clusters y B representa una de las clases de 'FORMAT', entonces el Lift está dado por:

$$\text{Lift} = \frac{P(B|A)}{P(B)} \tag{1}$$

Donde $P(B|A)$ es la proporción de la clase B dado que los datos pertenecen al cluster A y P(B) es la proporción global de la clase B. La forma de interpretar este índice es:

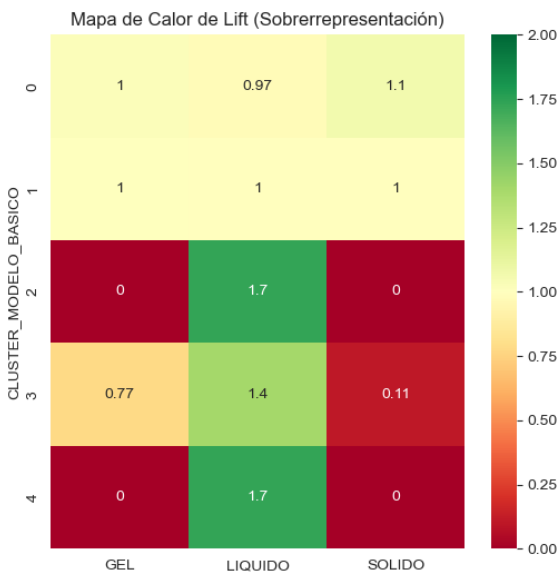
- **Lift > 1:** La clase está sobre-representada en ese grupo (es "especial" de ese grupo).
- **Lift = 1:** La clase se comporta igual que en el promedio general.
- **Lift < 1:** La clase está sub-representada.

```python
def heatmap_lift(df, col_clusters, categoria, figsize):
    # Crear la tabla de contingencia (Crosstab)
    counts = pd.crosstab(df[col_clusters], df[categoria])

    # Calcular proporciones dentro de cada grupo A
    prop_per_group = counts.div(counts.sum(axis=1), axis=0)

    # Calcular la proporción global de cada clase B
    global_prop = df[categoria].value_counts(normalize=True)

    # Calcular el LIFT
    # Dividimos la proporción del grupo entre la proporción global
    lift = prop_per_group.div(global_prop, axis=1)

    # Graficar
    plt.figure(figsize=figsize)
    sns.heatmap(lift, annot=True, cmap='RdYlGn', center=1, vmin=0, vmax=2)
    plt.title('Mapa de Calor de Lift (Sobrerrepresentación)')
    plt.show()
```

[10] ✓ 0.0s Python

```python
heatmap_lift(df=df2, col_clusters='CLUSTER_MODELO_BASICO', categoria='FORMAT', figsize=(6,6))
```

[11] ✓ 0.4s Python



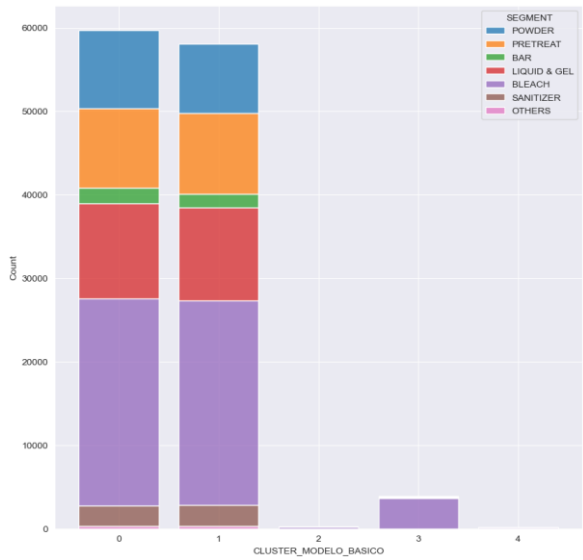
```python
resumen_clusters_basico.loc[[2,3,4],'Formato'] = 'LIQUIDO'
resumen_clusters_basico.loc[[0,1],'Formato'] = 'MIXTO'
resumen_clusters_basico
```

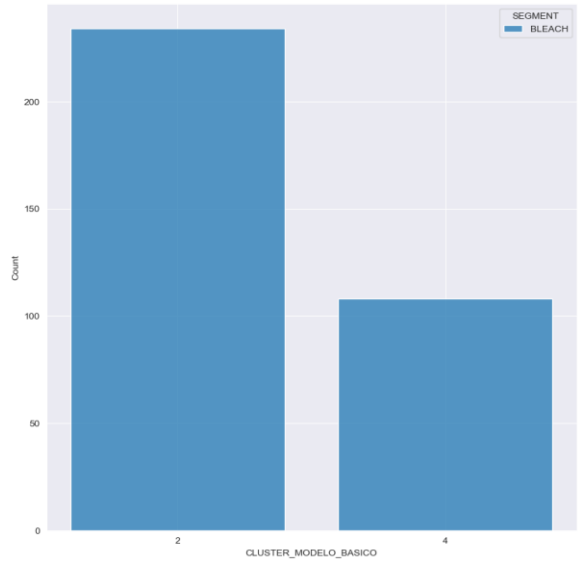
[12] ✓ 0.0s Python

Cluster	Representación del total	Regiones	Formato	Segmento	Ventas Unitarias Totales	Promedio de Ventas Unitarias	Fecha
0	48.9%	MIXTO	MIXTO	-	-	-	-
1	47.6%	MIXTO	MIXTO	-	-	-	-
2	0.2%	SCANNING MEXICO	LIQUIDO	-	-	-	-
3	3.2%	MIXTO	LIQUIDO	-	-	-	-
4	0.1%	AREA 3, SCANNING MEXICO	LIQUIDO	-	-	-	-

```python
df2 = graficar_barras_apiladas(df=df, col_clusters='CLUSTER_MODELO_BASICO', k=5, categoria='SEGMENT', figsize=(10,10))

#Gráfica anterior pero solo con los clusters que representan la minoría
graficar_barras_minoria(df=df, col_clusters='CLUSTER_MODELO_BASICO', clusters_minoria=[2,4], categoria='SEGMENT', figsize=(10,10))
```

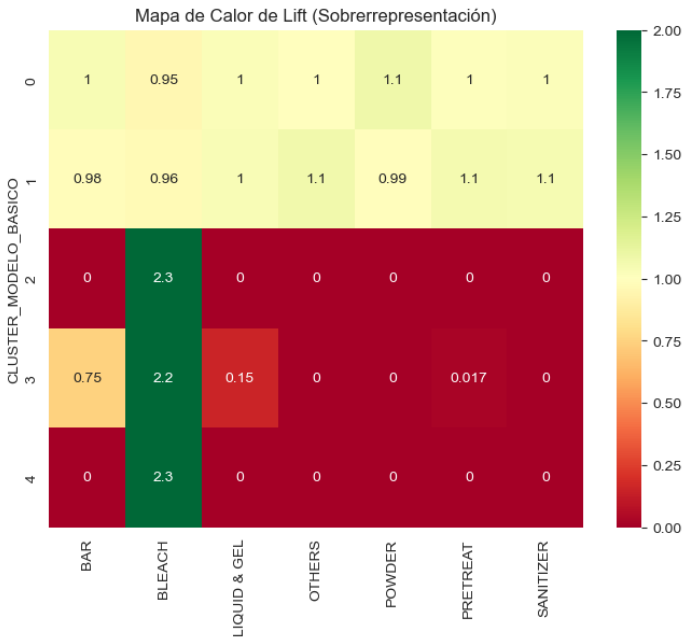
[13] ✓ 1.1s Python






```python
heatmap_lift(df=df2, col_clusters='CLUSTER_MODELO_BASICO', categoria='SEGMENT', figsize=(8,6))
```

[14] ✓ 0.5s Python



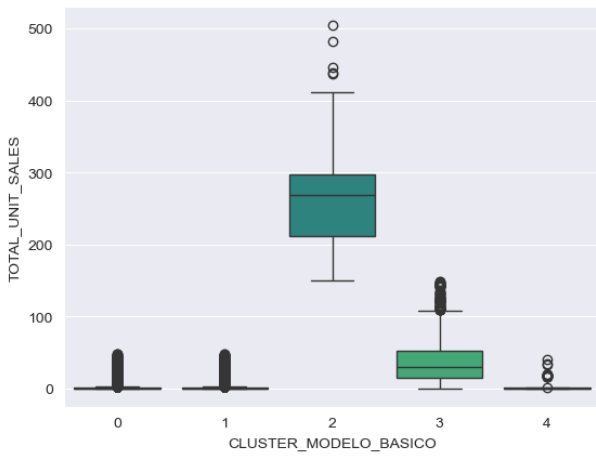
```python
resumen_clusters_basico.loc[[2,3,4],'Segmento'] = 'BLEACH'
resumen_clusters_basico.loc[[0,1],'Segmento'] = 'MIXTO'
resumen_clusters_basico
```

[15] ✓ 0.0s Python

Cluster	Representación del total	Regiones	Formato	Segmento	Ventas Unitarias Totales	Promedio de Ventas Unitarias	Fecha
0	48.9%	MIXTO	MIXTO	MIXTO	-	-	-
1	47.6%	MIXTO	MIXTO	MIXTO	-	-	-
2	0.2%	SCANNING MEXICO	LIQUIDO	BLEACH	-	-	-
3	3.2%	MIXTO	LIQUIDO	BLEACH	-	-	-
4	0.1%	AREA 3, SCANNING MEXICO	LIQUIDO	BLEACH	-	-	-

```python
sns.boxplot(data=df2, x='CLUSTER_MODELO_BASICO', y='TOTAL_UNIT_SALES', palette='viridis')
plt.show()

pd.pivot_table(df2, values='TOTAL_UNIT_SALES', columns='CLUSTER_MODELO_BASICO', aggfunc=[np.median])
```

[16] ✓ 0.5s Python



	median				
CLUSTER_MODELO_BASICO	0	1	2	3	4
TOTAL_UNIT_SALES	0.307	0.361	268.861	30.1385	0.45

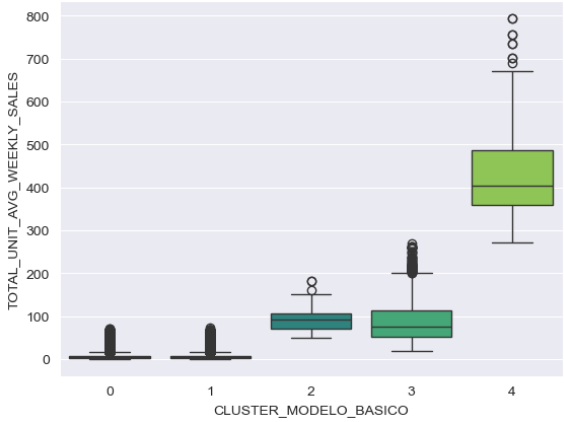
```python
resumen_clusters_basico.loc[2,'Ventas Unitarias Totales'] = 'VENTAS ALTAS (mediana=268)'
resumen_clusters_basico.loc[3,'Ventas Unitarias Totales'] = 'VENTAS MEDIAS (mediana=30)'
resumen_clusters_basico.loc[[0,1,4],'Ventas Unitarias Totales'] = 'VENTAS BAJAS (mediana<0.5)'
resumen_clusters_basico
```

[17] ✓ 0.0s Python

Cluster	Representación del total	Regiones	Formato	Segmento	Ventas Unitarias Totales	Promedio de Ventas Unitarias	Fecha
0	48.9%	MIXTO	MIXTO	MIXTO	VENTAS BAJAS (mediana<0.5)	-	-
1	47.6%	MIXTO	MIXTO	MIXTO	VENTAS BAJAS (mediana<0.5)	-	-
2	0.2%	SCANNING MEXICO	LIQUIDO	BLEACH	VENTAS ALTAS (mediana=268)	-	-
3	3.2%	MIXTO	LIQUIDO	BLEACH	VENTAS MEDIAS (mediana=30)	-	-
4	0.1%	AREA 3, SCANNING MEXICO	LIQUIDO	BLEACH	VENTAS BAJAS (mediana<0.5)	-	-

```python
sns.boxplot(data=df2, x='CLUSTER_MODELO_BASICO', y='TOTAL_UNIT_AVG_WEEKLY_SALES', palette='viridis')
plt.show()

pd.pivot_table(df2, values='TOTAL_UNIT_AVG_WEEKLY_SALES', columns='CLUSTER_MODELO_BASICO', aggfunc=[np.median])
```

[18] ✓ 0.6s Python



	median				
CLUSTER_MODELO_BASICO	0	1	2	3	4
TOTAL_UNIT_AVG_WEEKLY_SALES	3.723	3.8715	92.587	76.3225	404.5

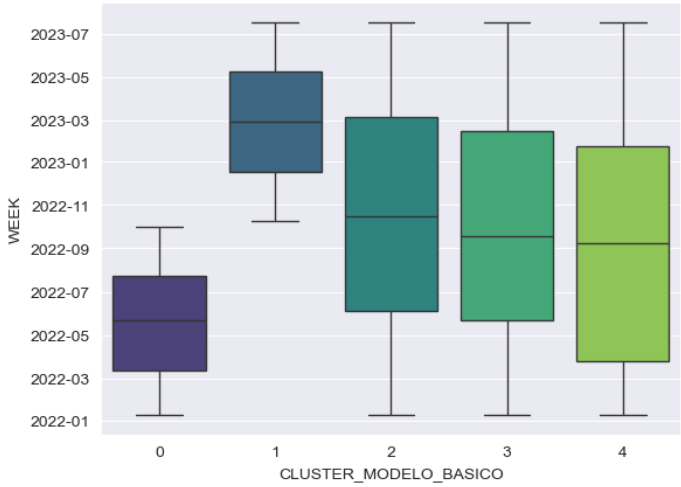
```python
resumen_clusters_basico.loc[4,'Promedio de Ventas Unitarias'] = 'VENTAS PROMEDIO ALTAS (mediana=405)'
resumen_clusters_basico.loc[[2,3],'Promedio de Ventas Unitarias'] = 'VENTAS PROMEDIO MEDIAS (75<mediana<95)'
resumen_clusters_basico.loc[[0,1],'Promedio de Ventas Unitarias'] = 'VENTAS PROMEDIO BAJAS (mediana<4)'
resumen_clusters_basico
```

✓ 0.0s Python

Cluster	Representación del total	Regiones	Formato	Segmento	Ventas Unitarias Totales	Promedio de Ventas Unitarias	Fecha
0	48.9%	MIXTO	MIXTO	MIXTO	VENTAS BAJAS (mediana<0.5)	VENTAS PROMEDIO BAJAS (mediana<4)	-
1	47.6%	MIXTO	MIXTO	MIXTO	VENTAS BAJAS (mediana<0.5)	VENTAS PROMEDIO BAJAS (mediana<4)	-
2	0.2%	SCANNING MEXICO	LIQUIDO	BLEACH	VENTAS ALTAS (mediana=268)	VENTAS PROMEDIO MEDIAS (75<mediana<95)	-
3	3.2%	MIXTO	LIQUIDO	BLEACH	VENTAS MEDIAS (mediana=30)	VENTAS PROMEDIO MEDIAS (75<mediana<95)	-
4	0.1%	AREA 3, SCANNING MEXICO	LIQUIDO	BLEACH	VENTAS BAJAS (mediana<0.5)	VENTAS PROMEDIO ALTAS (mediana=405)	-

```python
sns.boxplot(data=df2, x='CLUSTER_MODELO_BASICO', y='WEEK', palette='viridis')
plt.show()


pd.pivot_table(df2, values='WEEK', columns='CLUSTER_MODELO_BASICO', aggfunc=[np.min, np.max])
```

✓ 0.6s Python



	min					max				
CLUSTER_MODELO_BASICO	0	1	2	3	4	0	1	2	3	4
WEEK	2022-01-09	2022-10-09	2022-01-09	2022-01-09	2022-01-09	2022-10-02	2023-07-17	2023-07-17	2023-07-17	2023-07-17

```python
resumen_clusters_basico.loc[0,'Fecha'] = 'Invierno 2022 - Otoño 2022'
resumen_clusters_basico.loc[1,'Fecha'] = 'Otoño 2022 - Verano 2023'
resumen_clusters_basico.loc[[2,3,4],'Fecha'] = 'Todo el periodo (2022-2023)'
resumen_clusters_basico
```

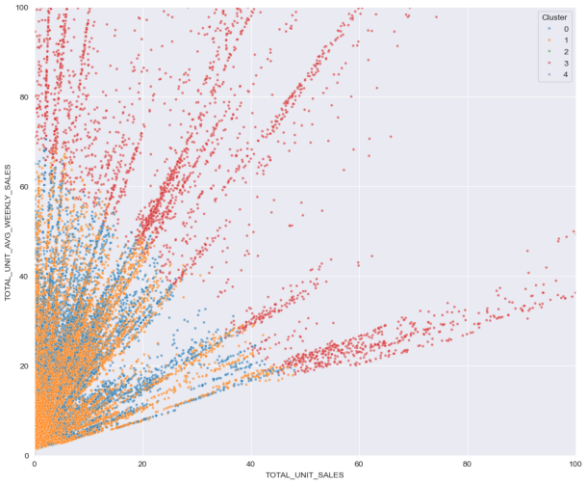
✓ 0.0s Python

Cluster	Representación del total	Regiones	Formato	Segmento	Ventas Unitarias Totales	Promedio de Ventas Unitarias	Fecha
0	48.9%	MIXTO	MIXTO	MIXTO	VENTAS BAJAS (mediana<0.5)	VENTAS PROMEDIO BAJAS (mediana<4)	Invierno 2022 - Otoño 2022
1	47.6%	MIXTO	MIXTO	MIXTO	VENTAS BAJAS (mediana<0.5)	VENTAS PROMEDIO BAJAS (mediana<4)	Otoño 2022 - Verano 2023
2	0.2%	SCANNING MEXICO	LIQUIDO	BLEACH	VENTAS ALTAS (mediana=268)	VENTAS PROMEDIO MEDIAS (75<mediana<95)	Todo el periodo (2022-2023)
3	3.2%	MIXTO	LIQUIDO	BLEACH	VENTAS MEDIAS (mediana=30)	VENTAS PROMEDIO MEDIAS (75<mediana<95)	Todo el periodo (2022-2023)
4	0.1%	AREA 3, SCANNING MEXICO	LIQUIDO	BLEACH	VENTAS BAJAS (mediana<0.5)	VENTAS PROMEDIO ALTAS (mediana=405)	Todo el periodo (2022-2023)

Finalmente, se mostrarán las distribuciones numéricas comparadas una contra otra en forma de gráficos de dispersión 2D.

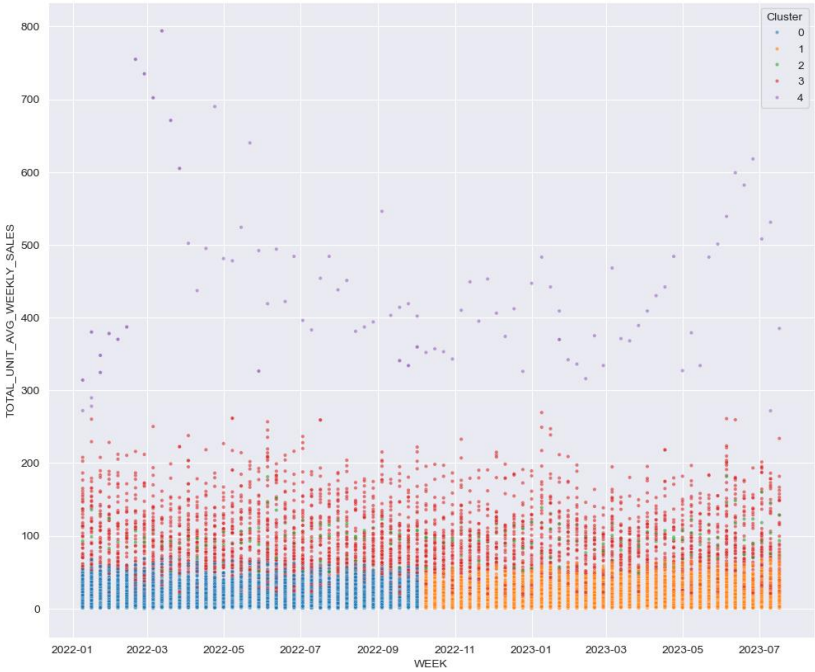
```python
f, ax = plt.subplots(figsize=(12,10))
sns.scatterplot(df2, x='TOTAL_UNIT_SALES', y='TOTAL_UNIT_AVG_WEEKLY_SALES', hue='CLUSTER_MODELO_BASICO', s=10)
plt.legend(title='Cluster')
plt.show()

# Gráfica con zoom
f, ax = plt.subplots(figsize=(12,10))
sns.scatterplot(df2, x='TOTAL_UNIT_SALES', y='TOTAL_UNIT_AVG_WEEKLY_SALES', hue='CLUSTER_MODELO_BASICO', s=10, alpha=0.6)
plt.xlim(0,100)
plt.ylim(0,100)
plt.legend(title='Cluster')
plt.show()
```

[21] ✓ 8.7s Python



```python
f, ax = plt.subplots(figsize=(12,10))
sns.scatterplot(df2, x='WEEK', y='TOTAL_UNIT_SALES', hue='CLUSTER_MODELO_BASICO', s=10, alpha=0.6)
plt.legend(title='Cluster')
plt.show()

f, ax = plt.subplots(figsize=(12,10))
sns.scatterplot(df2, x='WEEK', y='TOTAL_UNIT_AVG_WEEKLY_SALES', hue='CLUSTER_MODELO_BASICO', s=10, alpha=0.6)
plt.legend(title='Cluster')
plt.show()
```

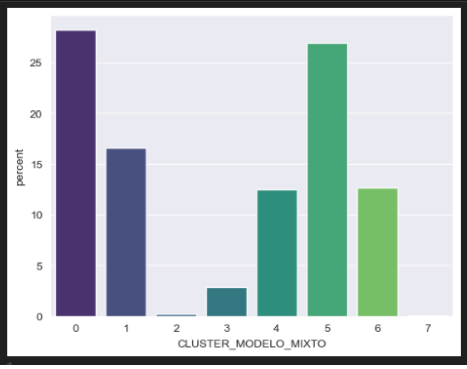
✓ 9.5s Python



Modelo mixto

```python
sns.countplot(data=df, x='CLUSTER_MODELO_MIXTO', palette='viridis', stat='percent')
plt.show()
```

0.6s Python




```python
(df['CLUSTER_MODELO_MIXTO'].value_counts() / len(df) * 100).sort_index()
```

0.0s Python

```
0    28.232324
1    16.562843
2     0.191800
3     2.904870
4    12.494877
5    26.908575
6    12.616187
7     0.088523
Name: CLUSTER_MODELO_MIXTO, dtype: float64
```

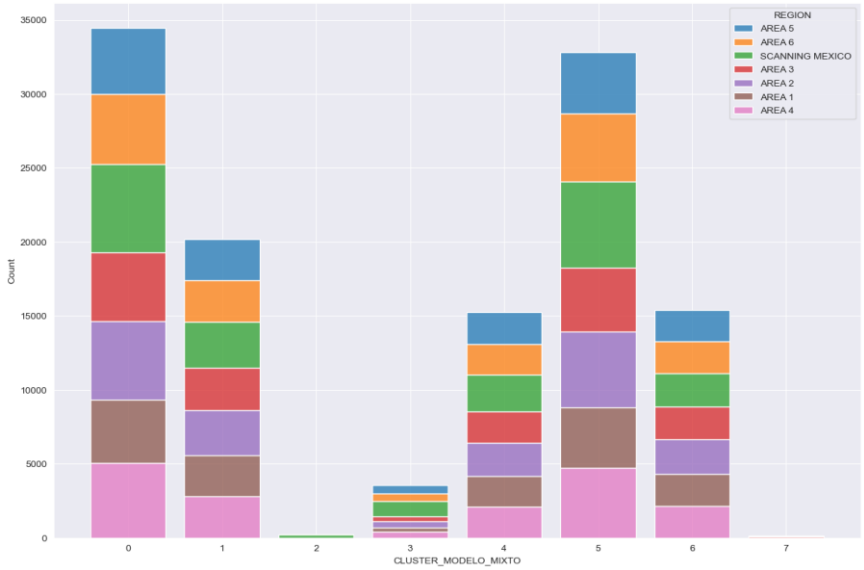
```python
# Creando un DataFrame para guardar el análisis de cada cluster
k=8
resumen_clusters_mixto = pd.DataFrame(data=np.full((k,7),fill_value='-'),
                                      columns=['Representación del total','Regiones','Formato','Segmento','Ventas Unitarias Totales','Promedio de Ventas Unitarias','Fecha'],
                                      index=np.arange(k))
resumen_clusters_mixto.index.name='Cluster'
resumen_clusters_mixto.loc[:,'Representación del total']=['28.2%', '16.6%', '0.2%', '2.9%', '12.5%', '26.9%', '12.6%', '0.1%']
resumen_clusters_mixto
```

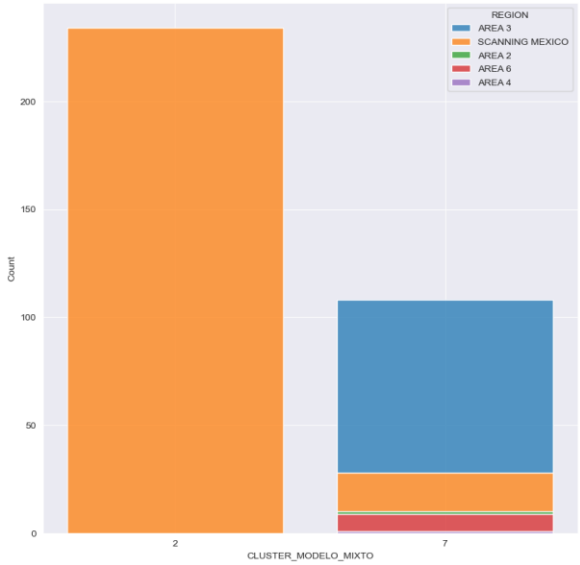
0.0s Python

Cluster	Representación del total	Regiones	Formato	Segmento	Ventas Unitarias Totales	Promedio de Ventas Unitarias	Fecha
0	28.2%	-	-	-	-	-	-
1	16.6%	-	-	-	-	-	-
2	0.2%	-	-	-	-	-	-
3	2.9%	-	-	-	-	-	-
4	12.5%	-	-	-	-	-	-
5	26.9%	-	-	-	-	-	-
6	12.6%	-	-	-	-	-	-
7	0.1%	-	-	-	-	-	-

```python
df2 = graficar_barras_apiladas(df=df2, col_clusters='CLUSTER_MODELO_MIXTO', k=8, categoria='REGION', figsize=(15,10))

#Gráfica anterior pero solo con los clusters que representan la minoría
graficar_barras_minoria(df=df, col_clusters='CLUSTER_MODELO_MIXTO', clusters_minoria=[2,7], categoria='REGION', figsize=(10,10))
```

1.3s Python


REGION
AREA 5
AREA 6
SCANNING MEXICO
AREA 3
AREA 2
AREA 1
AREA 4
Count
35000
30000
25000
20000
15000
10000
5000
0
0
1
2
3
4
5
6
7
CLUSTER_MODELO_MIXTO


REGION
AREA 3
SCANNING MEXICO
AREA 2
AREA 6
AREA 4
Count
200
150
100
50
0
2
7
CLUSTER_MODELO_MIXTO

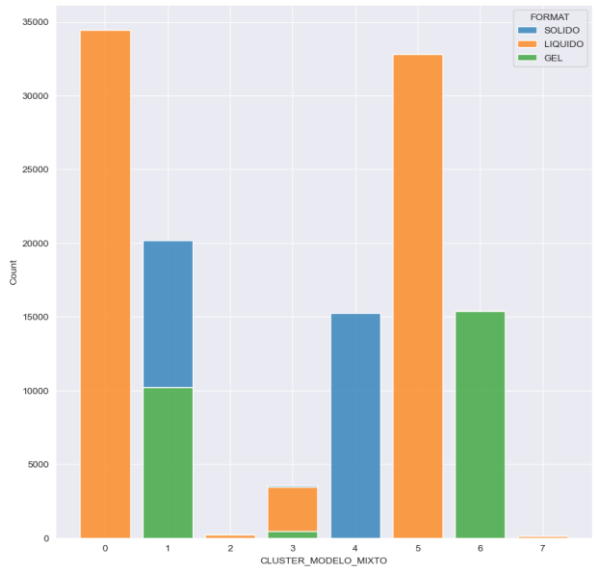
```python
resumen_clusters_mixto.loc[2,'Regiones']='SCANNING MEXICO'
resumen_clusters_mixto.loc[7,'Regiones']='AREA 3, SCANNING MEXICO'
resumen_clusters_mixto.loc[[0,1,3,4,5,6],'Regiones'] = 'MIXTO'
resumen_clusters_mixto
```

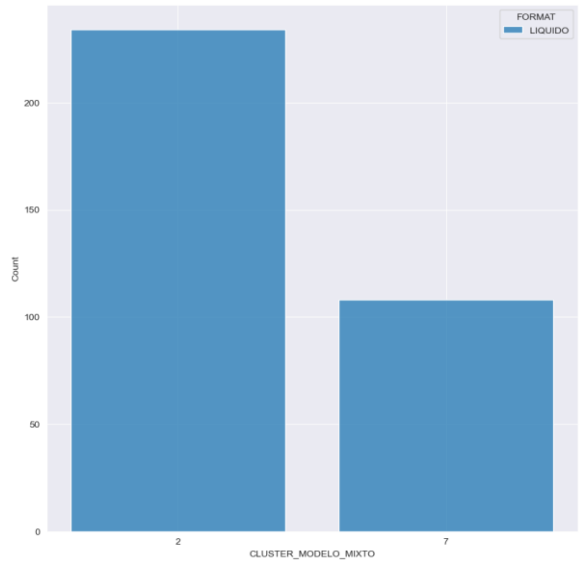
✓ 0.0s Python

Cluster	Representación del total	Regiones	Formato	Segmento	Ventas Unitarias Totales	Promedio de Ventas Unitarias	Fecha
0	28.2%	MIXTO	-	-	-	-	-
1	16.6%	MIXTO	-	-	-	-	-
2	0.2%	SCANNING MEXICO	-	-	-	-	-
3	2.9%	MIXTO	-	-	-	-	-
4	12.5%	MIXTO	-	-	-	-	-
5	26.9%	MIXTO	-	-	-	-	-
6	12.6%	MIXTO	-	-	-	-	-
7	0.1%	AREA 3, SCANNING MEXICO	-	-	-	-	-

```python
df2 = graficar_barras_apiladas(df=df2, col_clusters='CLUSTER_MODELO_MIXTO', k=8, categoria='FORMAT', figsize=(10,10))

#Gráfica anterior pero solo con los clusters que representan la minoría
graficar_barras_minoria(df=df, col_clusters='CLUSTER_MODELO_MIXTO', clusters_minoria=[2,7], categoria='FORMAT', figsize=(10,10))
```

✓ 0.9s Python






```python
heatmap_lift(df=df2, col_clusters='CLUSTER_MODELO_MIXTO', categoria='FORMAT', figsize=(5,6))
```

✓ 0.4s Python

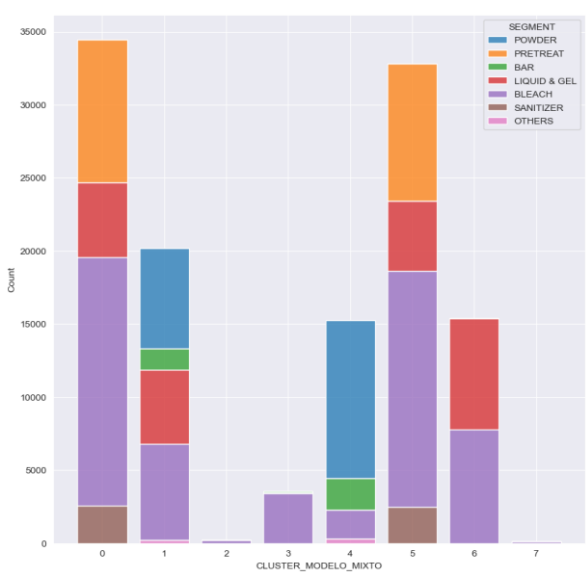
```
resumen_clusters_mixto.loc[1,'Formato'] = 'GEL, SOLIDO'
resumen_clusters_mixto.loc[4,'Formato'] = 'SOLIDO'
resumen_clusters_mixto.loc[6,'Formato'] = 'GEL'
resumen_clusters_mixto.loc[[0,2,3,5,7],'Formato'] = 'LIQUIDO'
resumen_clusters_mixto
```

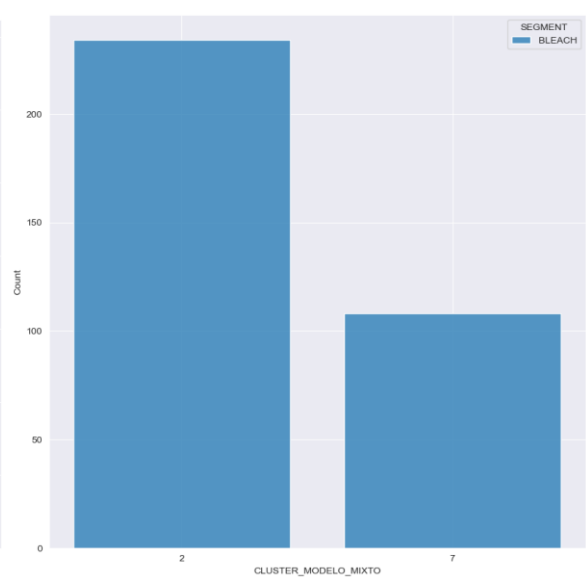
31] ✓ 0.0s Python

	Representación del total	Regiones	Formato	Segmento	Ventas Unitarias Totales	Promedio de Ventas Unitarias	Fecha
Cluster							
0	28.2%	MIXTO	LIQUIDO	-	-	-	-
1	16.6%	MIXTO	GEL, SOLIDO	-	-	-	-
2	0.2%	SCANNING MEXICO	LIQUIDO	-	-	-	-
3	2.9%	MIXTO	LIQUIDO	-	-	-	-
4	12.5%	MIXTO	SOLIDO	-	-	-	-
5	26.9%	MIXTO	LIQUIDO	-	-	-	-
6	12.6%	MIXTO	GEL	-	-	-	-
7	0.1%	AREA 3, SCANNING MEXICO	LIQUIDO	-	-	-	-

```
df2 = graficar_barras_apiladas(df=df2, col_clusters='CLUSTER_MODELO_MIXTO', k=8, categoria='SEGMENT', figsize=(10,10))

#Gráfica anterior pero solo con los clusters que representan la minoría
graficar_barras_minoria(df=df, col_clusters='CLUSTER_MODELO_MIXTO', clusters_minoria=[2,7], categoria='SEGMENT', figsize=(10,10))
```

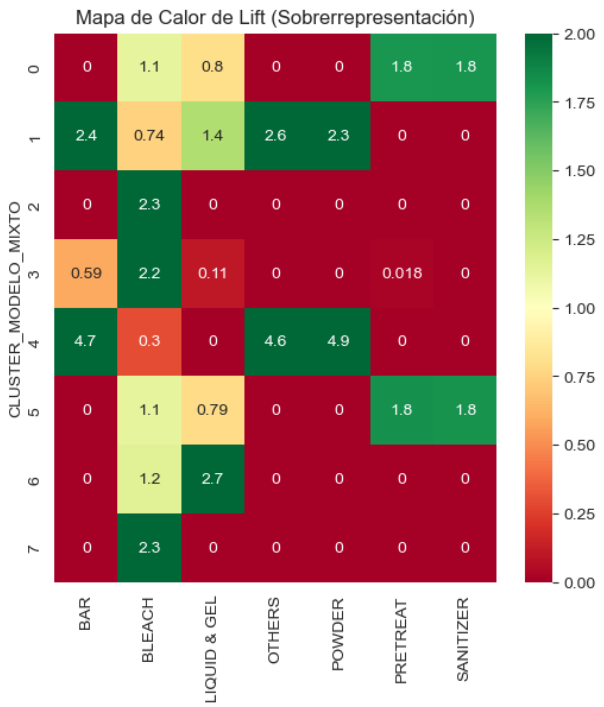
32] ✓ 1.1s Python





```python
heatmap_lift(df=df2, col_clusters='CLUSTER_MODELO_MIXTO', categoria='SEGMENT', figsize=(6,6))
```

[33] ✓ 0.7s Python



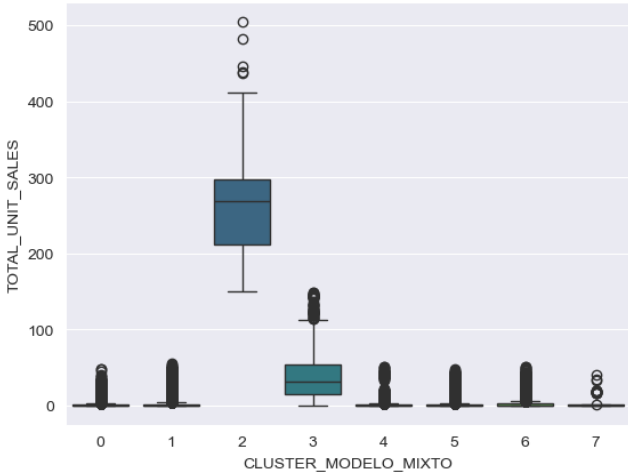

```python
resumen_clusters_mixto.loc[[2,3,7],'Segmento'] = 'BLEACH'
resumen_clusters_mixto.loc[6,'Segmento'] = 'LIQUID & GEL'
resumen_clusters_mixto.loc[[0,5],'Segmento'] = 'PRETREAT, SANITIZER'
resumen_clusters_mixto.loc[[1,4],'Segmento'] = 'BAR, OTHERS, POWDER'
resumen_clusters_mixto
```

[34] ✓ 0.0s Python

Cluster	Representación del total	Regiones	Formato	Segmento	Ventas Unitarias Totales	Promedio de Ventas Unitarias	Fecha
0	28.2%	MIXTO	LIQUIDO	PRETREAT, SANITIZER	-	-	-
1	16.6%	MIXTO	GEL, SOLIDO	BAR, OTHERS, POWDER	-	-	-
2	0.2%	SCANNING MEXICO	LIQUIDO	BLEACH	-	-	-
3	2.9%	MIXTO	LIQUIDO	BLEACH	-	-	-
4	12.5%	MIXTO	SOLIDO	BAR, OTHERS, POWDER	-	-	-
5	26.9%	MIXTO	LIQUIDO	PRETREAT, SANITIZER	-	-	-
6	12.6%	MIXTO	GEL	LIQUID & GEL	-	-	-
7	0.1%	AREA 3, SCANNING MEXICO	LIQUIDO	BLEACH	-	-	-

```python
sns.boxplot(data=df2, x='CLUSTER_MODELO_MIXTO', y='TOTAL_UNIT_SALES', palette='viridis')
plt.show()

pd.pivot_table(df2, values='TOTAL_UNIT_SALES', columns='CLUSTER_MODELO_MIXTO', aggfunc=[np.median])
```

[35] ✓ 0.6s Python



	median							
CLUSTER_MODELO_MIXTO	0	1	2	3	4	5	6	7
TOTAL_UNIT_SALES	0.2875	0.451	268.861	31.0405	0.2325	0.309	0.479	0.45

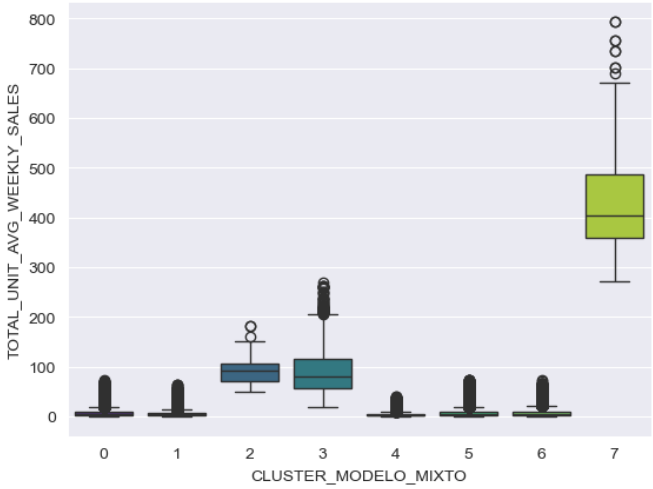
```
resumen_clusters_mixto.loc[2,'Ventas Unitarias Totales'] = 'VENTAS ALTAS (mediana=268)'
resumen_clusters_mixto.loc[3,'Ventas Unitarias Totales'] = 'VENTAS MEDIAS (mediana=31)'
resumen_clusters_mixto.loc[[0,1,4,5,6,7],'Ventas Unitarias Totales'] = 'VENTAS BAJAS (mediana<0.5)'
resumen_clusters_mixto
```

[36] ✓ 0.0s Python

Cluster	Representación del total	Regiones	Formato	Segmento	Ventas Unitarias Totales	Promedio de Ventas Unitarias	Fecha
0	28.2%	MIXTO	LIQUIDO	PRETREAT, SANITIZER	VENTAS BAJAS (mediana<0.5)	-	-
1	16.6%	MIXTO	GEL, SOLIDO	BAR, OTHERS, POWDER	VENTAS BAJAS (mediana<0.5)	-	-
2	0.2%	SCANNING MEXICO	LIQUIDO	BLEACH	VENTAS ALTAS (mediana=268)	-	-
3	2.9%	MIXTO	LIQUIDO	BLEACH	VENTAS MEDIAS (mediana=31)	-	-
4	12.5%	MIXTO	SOLIDO	BAR, OTHERS, POWDER	VENTAS BAJAS (mediana<0.5)	-	-
5	26.9%	MIXTO	LIQUIDO	PRETREAT, SANITIZER	VENTAS BAJAS (mediana<0.5)	-	-
6	12.6%	MIXTO	GEL	LIQUID & GEL	VENTAS BAJAS (mediana<0.5)	-	-
7	0.1%	AREA 3, SCANNING MEXICO	LIQUIDO	BLEACH	VENTAS BAJAS (mediana<0.5)	-	-

```
sns.boxplot(data=df2, x='CLUSTER_MODELO_MIXTO', y='TOTAL_UNIT_AVG_WEEKLY_SALES', palette='viridis')
plt.show()

pd.pivot_table(df2, values='TOTAL_UNIT_AVG_WEEKLY_SALES', columns='CLUSTER_MODELO_MIXTO', aggfunc=[np.median])
```

[37] ✓ 0.6s Python



	median							
CLUSTER_MODELO_MIXTO	0	1	2	3	4	5	6	7
TOTAL_UNIT_AVG_WEEKLY_SALES	4.333	3.521	92.587	81.0665	2.7715	4.234	4.007	404.5

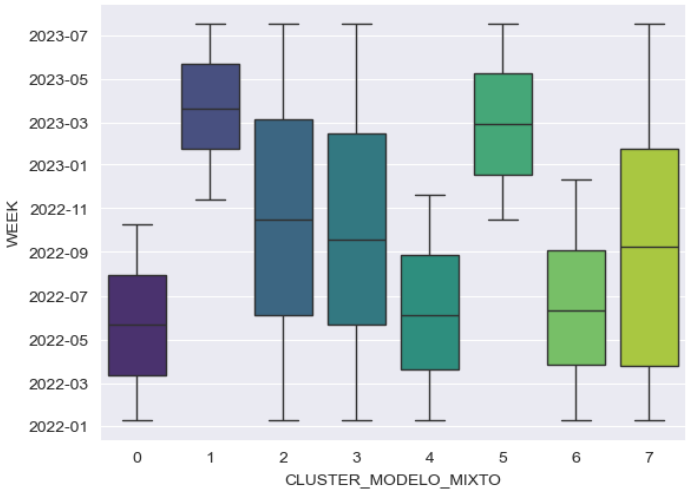
```python
resumen_clusters_mixto.loc[7,'Promedio de Ventas Unitarias'] = 'VENTAS PROMEDIO ALTAS (mediana=405)'
resumen_clusters_mixto.loc[[2,3],'Promedio de Ventas Unitarias'] = 'VENTAS PROMEDIO MEDIAS (75<mediana<95)'
resumen_clusters_mixto.loc[[0,1,4,5,6],'Promedio de Ventas Unitarias'] = 'VENTAS PROMEDIO BAJAS (mediana<5)'
resumen_clusters_mixto
```

[38] ✓ 0.0s Python

Cluster	Representación del total	Regiones	Formato	Segmento	Ventas Unitarias Totales	Promedio de Ventas Unitarias	Fecha
0	28.2%	MIXTO	LIQUIDO	PRETREAT, SANITIZER	VENTAS BAJAS (mediana<0.5)	VENTAS PROMEDIO BAJAS (mediana<5)	-
1	16.6%	MIXTO	GEL, SOLIDO	BAR, OTHERS, POWDER	VENTAS BAJAS (mediana<0.5)	VENTAS PROMEDIO BAJAS (mediana<5)	-
2	0.2%	SCANNING MEXICO	LIQUIDO	BLEACH	VENTAS ALTAS (mediana=268)	VENTAS PROMEDIO MEDIAS (75<mediana<95)	-
3	2.9%	MIXTO	LIQUIDO	BLEACH	VENTAS MEDIAS (mediana=31)	VENTAS PROMEDIO MEDIAS (75<mediana<95)	-
4	12.5%	MIXTO	SOLIDO	BAR, OTHERS, POWDER	VENTAS BAJAS (mediana<0.5)	VENTAS PROMEDIO BAJAS (mediana<5)	-
5	26.9%	MIXTO	LIQUIDO	PRETREAT, SANITIZER	VENTAS BAJAS (mediana<0.5)	VENTAS PROMEDIO BAJAS (mediana<5)	-
6	12.6%	MIXTO	GEL	LIQUID & GEL	VENTAS BAJAS (mediana<0.5)	VENTAS PROMEDIO BAJAS (mediana<5)	-
7	0.1%	AREA 3, SCANNING MEXICO	LIQUIDO	BLEACH	VENTAS BAJAS (mediana<0.5)	VENTAS PROMEDIO ALTAS (mediana=405)	-

```python
sns.boxplot(data=df2, x='CLUSTER_MODELO_MIXTO', y='WEEK', palette='viridis')
plt.show()

pd.pivot_table(df2, values='WEEK', columns='CLUSTER_MODELO_MIXTO', aggfunc=[np.min, np.max])
```

[39] ✓ 0.7s Python



	min								max							
CLUSTER_MODELO_MIXTO	0	1	2	3	4	5	6	7	0	1	2	3	4	5	6	7
WEEK	2022-01-09	2022-11-13	2022-01-09	2022-01-09	2022-01-09	2022-10-16	2022-01-09	2022-01-09	2022-10-09	2023-07-17	2023-07-17	2023-07-17	2022-11-20	2023-07-17	2022-12-11	2023-07-17

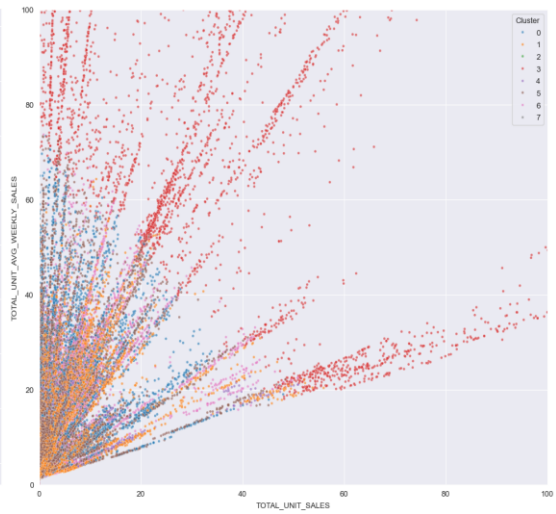
```python
resumen_clusters_mixto.loc[[0,4],'Fecha'] = 'Invierno 2022 - Otoño 2022'
resumen_clusters_mixto.loc[[1,5],'Fecha'] = 'Otoño 2022 - Verano 2023'
resumen_clusters_mixto.loc[[2,3,6,7],'Fecha'] = 'Todo el periodo (2022-2023)'
resumen_clusters_mixto
```

[40] ✓ 0.0s Python

Cluster	Representación del total	Regiones	Formato	Segmento	Ventas Unitarias Totales	Promedio de Ventas Unitarias	Fecha
0	28.2%	MIXTO	LIQUIDO	PRETREAT, SANITIZER	VENTAS BAJAS (mediana<0.5)	VENTAS PROMEDIO BAJAS (mediana<5)	Invierno 2022 - Otoño 2022
1	16.6%	MIXTO	GEL, SOLIDO	BAR, OTHERS, POWDER	VENTAS BAJAS (mediana<0.5)	VENTAS PROMEDIO BAJAS (mediana<5)	Otoño 2022 - Verano 2023
2	0.2%	SCANNING MEXICO	LIQUIDO	BLEACH	VENTAS ALTAS (mediana=268)	VENTAS PROMEDIO MEDIAS (75<mediana<95)	Todo el periodo (2022-2023)
3	2.9%	MIXTO	LIQUIDO	BLEACH	VENTAS MEDIAS (mediana=31)	VENTAS PROMEDIO MEDIAS (75<mediana<95)	Todo el periodo (2022-2023)
4	12.5%	MIXTO	SOLIDO	BAR, OTHERS, POWDER	VENTAS BAJAS (mediana<0.5)	VENTAS PROMEDIO BAJAS (mediana<5)	Invierno 2022 - Otoño 2022
5	26.9%	MIXTO	LIQUIDO	PRETREAT, SANITIZER	VENTAS BAJAS (mediana<0.5)	VENTAS PROMEDIO BAJAS (mediana<5)	Otoño 2022 - Verano 2023
6	12.6%	MIXTO	GEL	LIQUID & GEL	VENTAS BAJAS (mediana<0.5)	VENTAS PROMEDIO BAJAS (mediana<5)	Todo el periodo (2022-2023)
7	0.1%	AREA 3, SCANNING MEXICO	LIQUIDO	BLEACH	VENTAS BAJAS (mediana<0.5)	VENTAS PROMEDIO ALTAS (mediana=405)	Todo el periodo (2022-2023)

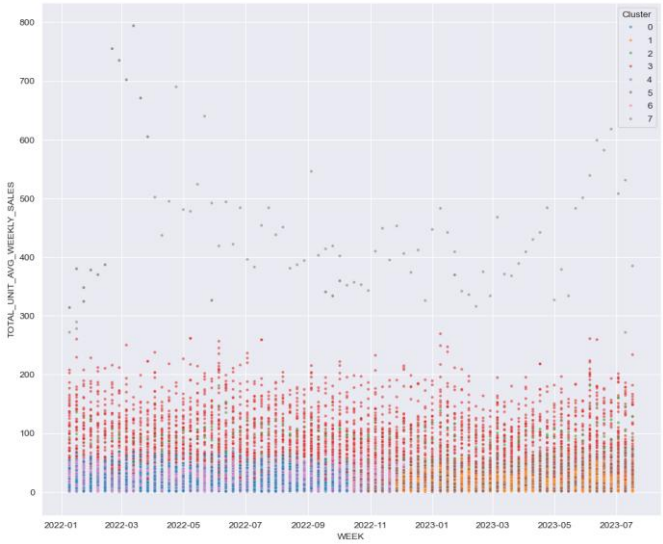
```python
f, ax = plt.subplots(figsize=(12,10))
sns.scatterplot(df2, x='TOTAL_UNIT_SALES', y='TOTAL_UNIT_AVG_WEEKLY_SALES', hue='CLUSTER_MODELO_MIXTO', s=10)
plt.legend(title='Cluster')
plt.show()

# Gráfica con zoom
f, ax = plt.subplots(figsize=(12,10))
sns.scatterplot(df2, x='TOTAL_UNIT_SALES', y='TOTAL_UNIT_AVG_WEEKLY_SALES', hue='CLUSTER_MODELO_MIXTO', s=10, alpha=0.6)
plt.xlim(0,100)
plt.ylim(0,100)
plt.legend(title='Cluster')
plt.show()
```

✓ 8.7s Python




```python
f, ax = plt.subplots(figsize=(12,10))
sns.scatterplot(df2, x='WEEK', y='TOTAL_UNIT_SALES', hue='CLUSTER_MODELO_MIXTO', s=10, alpha=0.6)
plt.legend(title='Cluster')
plt.show()

f, ax = plt.subplots(figsize=(12,10))
sns.scatterplot(df2, x='WEEK', y='TOTAL_UNIT_AVG_WEEKLY_SALES', hue='CLUSTER_MODELO_MIXTO', s=10, alpha=0.6)
plt.legend(title='Cluster')
plt.show()
```

✓ 9.3s Python


Cluster
0
1
2
3
4
5
6
7
800
700
600
500
400
300
200
100
0
TOTAL_UNIT_AVG_WEEKLY_SALES
2022-01
2022-03
2022-05
2022-07
2022-09
2022-11
2023-01
2023-03
2023-05
2023-07
WEEK

Comparación de los dos modelos

```python
resumen_clusters_basico
```

[43] 0.0s Python

Cluster	Representación del total	Regiones	Formato	Segmento	Ventas Unitarias Totales	Promedio de Ventas Unitarias	Fecha
0	48.9%	MIXTO	MIXTO	MIXTO	VENTAS BAJAS (mediana<0.5)	VENTAS PROMEDIO BAJAS (mediana<4)	Invierno 2022 - Otoño 2022
1	47.6%	MIXTO	MIXTO	MIXTO	VENTAS BAJAS (mediana<0.5)	VENTAS PROMEDIO BAJAS (mediana<4)	Otoño 2022 - Verano 2023
2	0.2%	SCANNING MEXICO	LIQUIDO	BLEACH	VENTAS ALTAS (mediana=268)	VENTAS PROMEDIO MEDIAS (75<mediana<95)	Todo el periodo (2022-2023)
3	3.2%	MIXTO	LIQUIDO	BLEACH	VENTAS MEDIAS (mediana=30)	VENTAS PROMEDIO MEDIAS (75<mediana<95)	Todo el periodo (2022-2023)
4	0.1%	AREA 3, SCANNING MEXICO	LIQUIDO	BLEACH	VENTAS BAJAS (mediana<0.5)	VENTAS PROMEDIO ALTAS (mediana=405)	Todo el periodo (2022-2023)

```python
resumen_clusters_mixto
```

[44] 0.0s Python

Cluster	Representación del total	Regiones	Formato	Segmento	Ventas Unitarias Totales	Promedio de Ventas Unitarias	Fecha
0	28.2%	MIXTO	LIQUIDO	PRETREAT, SANITIZER	VENTAS BAJAS (mediana<0.5)	VENTAS PROMEDIO BAJAS (mediana<5)	Invierno 2022 - Otoño 2022
1	16.6%	MIXTO	GEL, SOLIDO	BAR, OTHERS, POWDER	VENTAS BAJAS (mediana<0.5)	VENTAS PROMEDIO BAJAS (mediana<5)	Otoño 2022 - Verano 2023
2	0.2%	SCANNING MEXICO	LIQUIDO	BLEACH	VENTAS ALTAS (mediana=268)	VENTAS PROMEDIO MEDIAS (75<mediana<95)	Todo el periodo (2022-2023)
3	2.9%	MIXTO	LIQUIDO	BLEACH	VENTAS MEDIAS (mediana=31)	VENTAS PROMEDIO MEDIAS (75<mediana<95)	Todo el periodo (2022-2023)
4	12.5%	MIXTO	SOLIDO	BAR, OTHERS, POWDER	VENTAS BAJAS (mediana<0.5)	VENTAS PROMEDIO BAJAS (mediana<5)	Invierno 2022 - Otoño 2022
5	26.9%	MIXTO	LIQUIDO	PRETREAT, SANITIZER	VENTAS BAJAS (mediana<0.5)	VENTAS PROMEDIO BAJAS (mediana<5)	Otoño 2022 - Verano 2023
6	12.6%	MIXTO	GEL	LIQUID & GEL	VENTAS BAJAS (mediana<0.5)	VENTAS PROMEDIO BAJAS (mediana<5)	Todo el periodo (2022-2023)
7	0.1%	AREA 3, SCANNING MEXICO	LIQUIDO	BLEACH	VENTAS BAJAS (mediana<0.5)	VENTAS PROMEDIO ALTAS (mediana=405)	Todo el periodo (2022-2023)

Se puede apreciar que en ambos modelos, el cluster 2 es exactamente el mismo; y el cluster 4 del modelo básico es prácticamente el mismo cluster 7 encontrado por el modelo mixto. El cluster 3 en ambos modelos son muy parecidos en todas sus características, a excepción de que en el modelo básico representan el 3.2% y en el mixto representan el 2.9%, por lo que algunos datos de ventas en el modelo mixto fueron distribuidos en otros clusters. Además de esto, los clusters 0 y 1 del modelo básico fueron divididos en los demás clusters encontrados por el modelo mixto, aparenciendo una división más clara en los formatos y segmentos de los productos.

¿Cuál será el modelo final a analizar?

Por cuestiones prácticas y por la mayor información proveída por el modelo mixto para cada uno de los clusters, este será el modelo final a analizar por el resto de este proyecto.

```python
df.drop(columns=['CLUSTER_MODELO_BASICO'], inplace=True) #Eliminando la columna de los clusters obtenidos con el modelo básico
df.rename(columns={'CLUSTER_MODELO_MIXTO':'CLUSTER'}, inplace=True) #Renombrando la columna del modelo mixto a un nombre más simple
df
```

[46] 0.0s Python

	WEEK	TOTAL_UNIT_SALES	TOTAL_VALUE_SALES	TOTAL_UNIT_AVG_WEEKLY_SALES	REGION	MANUFACTURER	BRAND	ITEM_DESCRIPTION	FORMAT
0	2022-08-28	0.006	0.139	1.000	AREA 5	RECKITT	VANISH	VANISHOXIACTIONROSADOYPACK120GR+MMCRYSTALWHITE...	SOLIDO
1	2022-08-28	0.487	116.519	2.916	AREA 5	RECKITT	VANISH	VANISH OXI ACTION GOLD QUITAMANCHAS BOLSA 1.8K...	SOLIDO
2	2022-08-28	1.391	68.453	5.171	AREA 5	RECKITT	VANISH	VANISH OXI ACTION ROSA QUITAMANCHAS DOYPACK 24...	SOLIDO
3	2022-08-28	0.022	1.481	1.833	AREA 5	RECKITT	VANISH	VANISH OXI ACTION GOLD QUITAMANCHA AHORRO DEL ...	SOLIDO
4	2022-08-28	2.037	182.839	5.375	AREA 5	RECKITT	VANISH	VANISH INTELLIGENCE POLVO BOTE 450 GR NAL 7501...	SOLIDO
...	...	...	...	...	...	...	...	...	...
121997	2023-07-03	1.192	150.829	4.000	SCANNING MEXICO	RECKITT	VANISH	VANISHOXIACTIONROSABOTE900GR+MMCRYSTALWHITEDOY...	SOLIDO
121998	2023-07-03	1.947	235.416	2.797	SCANNING MEXICO	RECKITT	VANISH	VANISH PODER O2 PRE-LAVADOR BOT.ATOM. 650 ML N...	LIQUIDO
121999	2023-07-03	18.013	429.289	9.102	SCANNING MEXICO	RECKITT	VANISH	VANISH PODER O2 DOYPACK 120 GR 7501058753441	SOLIDO
122000	2023-07-03	21.152	210.276	22.172	SCANNING MEXICO	RECKITT	VANISH	VANISH PODER 02 INTELIGENCE SIN CLORO SOBRE 30...	SOLIDO

Resultados e Insights clave

Asignación de nombres a clusters

Vamos a volver a visulaizar el resumen con las características de los clusters obtenidos con el modelo final, para asignarles un nombre clave a cada uno de estos

```python
resumen_clusters_mixto
```

[46] ✓ 0.0s Python

Cluster	Representación del total	Regiones	Formato	Segmento	Ventas Unitarias Totales	Promedio de Ventas Unitarias	Fecha
0	28.2%	MIXTO	LIQUIDO	PRETREAT, SANITIZER	VENTAS BAJAS (mediana<0.5)	VENTAS PROMEDIO BAJAS (mediana<5)	Invierno 2022 - Otoño 2022
1	16.6%	MIXTO	GEL, SOLIDO	BAR, OTHERS, POWDER	VENTAS BAJAS (mediana<0.5)	VENTAS PROMEDIO BAJAS (mediana<5)	Otoño 2022 - Verano 2023
2	0.2%	SCANNING MEXICO	LIQUIDO	BLEACH	VENTAS ALTAS (mediana=268)	VENTAS PROMEDIO MEDIAS (75<mediana<95)	Todo el periodo (2022-2023)
3	2.9%	MIXTO	LIQUIDO	BLEACH	VENTAS MEDIAS (mediana=31)	VENTAS PROMEDIO MEDIAS (75<mediana<95)	Todo el periodo (2022-2023)
4	12.5%	MIXTO	SOLIDO	BAR, OTHERS, POWDER	VENTAS BAJAS (mediana<0.5)	VENTAS PROMEDIO BAJAS (mediana<5)	Invierno 2022 - Otoño 2022
5	26.9%	MIXTO	LIQUIDO	PRETREAT, SANITIZER	VENTAS BAJAS (mediana<0.5)	VENTAS PROMEDIO BAJAS (mediana<5)	Otoño 2022 - Verano 2023
6	12.6%	MIXTO	GEL	LIQUID & GEL	VENTAS BAJAS (mediana<0.5)	VENTAS PROMEDIO BAJAS (mediana<5)	Todo el periodo (2022-2023)
7	0.1%	AREA 3, SCANNING MEXICO	LIQUIDO	BLEACH	VENTAS BAJAS (mediana<0.5)	VENTAS PROMEDIO ALTAS (mediana=405)	Todo el periodo (2022-2023)

Analizando de nuevo las características de cada cluster, se decidió colocar los siguientes nombres a cada cluster:

- **Cluster 0 - Seasonal Low-Sales Liquid Treatments (2022)**: Productos de limpieza líquida con bajas ventas específicos del periodo 2022.
- **Cluster 1 - Low-Sales Solids & Gels (2023)**: Barras y polvos con ventas bajas, concentrados principalmente en 2023.
- **Cluster 2 - High-Performance Regional Bleach (Scanning Mexico)**: Es el clúster estrella, pues representa productos de cloro líquido con ventas muy altas y que superan el promedio de ventas semanales histórico, en la región de Scanning Mexico.
- **Cluster 3 - Stable Mid-Sales Liquid Bleach**: Ventas estables y moderadas de cloro líquido en diversas regiones.
- **Cluster 4 - Low-Sales Solid Cleaners (2022)**: Formatos sólidos (barras/polvos) con bajo desempeño durante el año 2022.
- **Cluster 5 - Seasonal Low-Sales Liquid Treatments (2023)**: Similar al cluster 0, pero representa la tendencia de bajo rendimiento en ventas en el periodo 2023
- **Cluster 6 - Continuous Low-Demand Gels**: Productos en gel que mantienen ventas bajas de manera constante en todo el periodo.
- **Cluster 7 - High-Potential Niche Regional Bleach**: Grupo muy pequeño con ventas actuales bajas pero con un promedio histórico altísimo en regiones específicas (Area 3 y Scanning Mexico).

```python
df['CLUSTER_NAME']=df['CLUSTER'].copy()
# Vamos a agregar una columna en la que se encuentre guardado el nombre del cluster, lo cual será útil para futuros gráficos
df['CLUSTER_NAME'] = df['CLUSTER_NAME'].replace(to_replace=[0,1,2,3,4,5,6,7],
                                                value=['Seasonal Low-Sales Liquid Treatments (2022)', 'Low-Sales Solids & Gels (2023)',
                                                       'High-Performance Regional Bleach (Scanning Mexico)', 'Stable Mid-Sales Liquid Bleach',
                                                       'Low-Sales Solid Cleaners (2022)', 'Seasonal Low-Sales Liquid Treatments (2023)',
                                                       'Continuous Low-Demand Gels', 'High-Potential Niche Regional Bleach'])
df
```

[47] ✓ 0.0s Python

LES	REGION	MANUFACTURER	BRAND	ITEM_DESCRIPTION	FORMAT	ATTR1	ATTR2	ATTR3	SEGMENT	WEEK_HOURS	CLUSTER	CLUSTER_NAME
000	AREA 5	RECKITT	VANISH	VANISHOXIACTIONROSADOYPACK120GR+MMCRYSTALWHITE...	SOLIDO	SAFE BLEACH	FABRIC TREATMENT	ROSA	POWDER	461568	4	Low-Sales Solid Cleaners (2022)
916	AREA 5	RECKITT	VANISH	VANISH OXI ACTION GOLD QUITAMANCHAS BOLSA 1.8K...	SOLIDO	SAFE BLEACH	FABRIC TREATMENT	ROSA	POWDER	461568	4	Low-Sales Solid Cleaners (2022)
171	AREA 5	RECKITT	VANISH	VANISH OXI ACTION ROSA QUITAMANCHAS DOYPACK 24...	SOLIDO	SAFE BLEACH	FABRIC TREATMENT	ROSA	POWDER	461568	4	Low-Sales Solid Cleaners (2022)
833	AREA 5	RECKITT	VANISH	VANISH OXI ACTION GOLD QUITAMANCHA AHORRO DEL ...	SOLIDO	SAFE BLEACH	FABRIC TREATMENT	ROSA	POWDER	461568	4	Low-Sales Solid Cleaners (2022)
375	AREA 5	RECKITT	VANISH	VANISH INTELLIGENCE POLVO BOTE 450 GR NAL 7501...	SOLIDO	SAFE BLEACH	FABRIC TREATMENT	ROSA	POWDER	461568	4	Low-Sales Solid Cleaners (2022)
...	...	...	...	...	...	...	...	...	...	...	...	...
000	SCANNING MEXICO	RECKITT	VANISH	VANISHOXIACTIONROSABOTE900GR+MMCRYSTALWHITEDOY...	SOLIDO	SAFE BLEACH	FABRIC TREATMENT	ROSA	POWDER	468984	1	Low-Sales Solids & Gels (2023)
797	SCANNING MEXICO	RECKITT	VANISH	VANISH PODER O2 PRE-LAVADOR BOT.ATOM. 650 ML N...	LIQUIDO	PRELAVADOR	FABRIC TREATMENT	ROSA	PRETREAT	468984	5	Seasonal Low-Sales Liquid Treatments (2023)
102	SCANNING MEXICO	RECKITT	VANISH	VANISH PODER O2 DOYPACK 120 GR 7501058753441	SOLIDO	SAFE BLEACH	FABRIC TREATMENT	ROSA	POWDER	468984	1	Low-Sales Solids & Gels (2023)
172	SCANNING MEXICO	RECKITT	VANISH	VANISH PODER 02 INTELIGENCE SIN CLORO SOBRE 30...	SOLIDO	SAFE BLEACH	FABRIC TREATMENT	ROSA	POWDER	468984	1	Low-Sales Solids & Gels (2023)
068	SCANNING MEXICO	RECKITT	VANISH	VANISH PODER 02 INTELLIGENCE POLVO BOT 900 GR ...	SOLIDO	SAFE BLEACH	FABRIC TREATMENT	ROSA	POWDER	468984	1	Low-Sales Solids & Gels (2023)

Análisis de clusters por rendimiento

Los clusters 2 y 3 representan productos que suelen tener un promedio de ventas semanales medio, diferenciándose porque el primero tuvo un mejor y muy alto rendimiento en ventas y el segundo tuvo ventas semanales medias (siendo igual a lo usual); ambos durante todo el periodo registrado (2022-2023). El formato y segmento representativo en estos clusters son los mismos: productos de cloro líquido, siendo la principal diferencia que el cluster de rendimiento alto representa solo la región de Scanning Mexico y el de rendimiento medio es una mezcla de todas las regiones.

Por otro lado, podemos ver que los clusters 0, 1, 4, 5, 6 tienen un muy bajo rendimiento tanto en ventas como en su promedio histórico. Estos clusters no tienen una región definida y solo están separados por fechas, formatos y segmentos. En cambio, el cluster 7 se le considera de "alto potencial", debido a que sus ventas actuales fueron bajas, pero su promedio histórico es muy alto, principalmente en regiones como Area 3 y Scanning Mexico.

De lo anterior, lo más relevante a notar son las características que resaltan de los clusters 2 ("High-Performance Regional Bleach (Scanning Mexico)"), 3 ("Stable Mid-Sales Liquid Bleach") y 7 ("High-Potential Niche Regional Bleach"), que representan una minoría importante de todos los datos (un 3.2% sumando la contibución de los tres clusters), es lo siguiente: los productos de cloro líquido son aquellos que llegan a destacar por su buen rendimiento, lo cual se aprecia principalmente en la región de Scanning Mexico. En esta misima región y en el Area 3 se detectó que hay un alto potencial. A continuación, se analizará a produndidad estos clusters, para encontrar si existen patrones en otras características (como en sus atributos y marcas) que expliquen estas altas ventas semanales y promedios históricos.

Análisis de atributos

Análisis de clusters por rendimiento

Los clusters 2 y 3 representan productos que suelen tener un promedio de ventas semanales medio, diferenciándose porque el primero tuvo un mejor y muy alto rendimiento en ventas y el segundo tuvo ventas semanales medias (siendo igual a lo usual); ambos durante todo el periodo registrado (2022-2023). El formato y segmento representativo en estos clusters son los mismos: productos de cloro líquido, siendo la principal diferencia que el cluster de rendimiento alto representa solo la región de Scanning Mexico y el de rendimiento medio es una mezcla de todas las regiones.

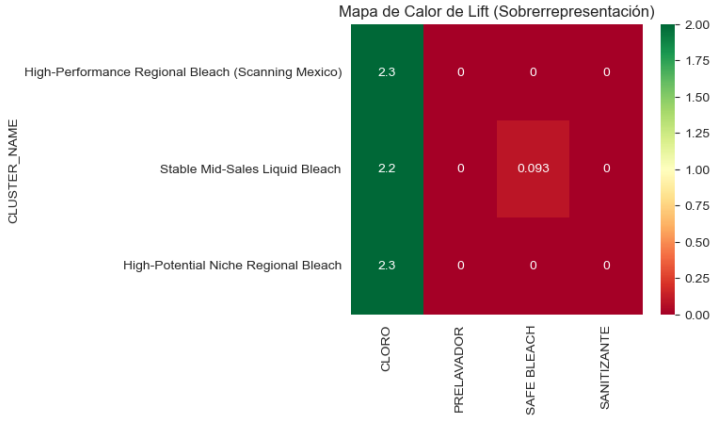
Por otro lado, podemos ver que los clusters 0, 1, 4, 5, 6 tienen un muy bajo rendimiento tanto en ventas como en su promedio histórico. Estos clusters no tienen una región definida y solo están separados por fechas, formatos y segmentos. En cambio, el cluster 7 se le considera de "alto potencial", debido a que sus ventas actuales fueron bajas, pero su promedio histórico es muy alto, principalmente en regiones como Area 3 y Scanning Mexico.

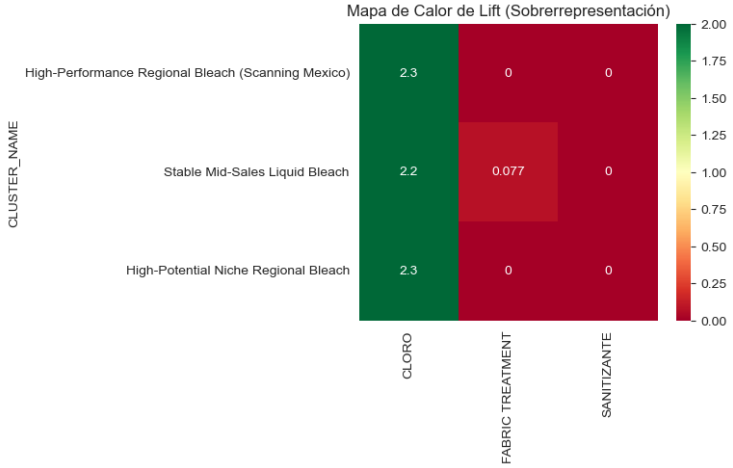
De lo anterior, lo más relevante a notar son las características que resaltan de los clusters 2 ("High-Performance Regional Bleach (Scanning Mexico)"), 3 ("Stable Mid-Sales Liquid Bleach") y 7 ("High-Potential Niche Regional Bleach"), que representan una minoría importante de todos los datos (un 3.2% sumando la contibución de los tres clusters), es lo siguiente: los productos de cloro líquido son aquellos que llegan a destacar por su buen rendimiento, lo cual se aprecia principalmente en la región de Scanning Mexico. En esta misima región y en el Area 3 se detectó que hay un alto potencial. A continuación, se analizará a produndidad estos clusters, para encontrar si existen patrones en otras características (como en sus atributos y marcas) que expliquen estas altas ventas semanales y promedios históricos.

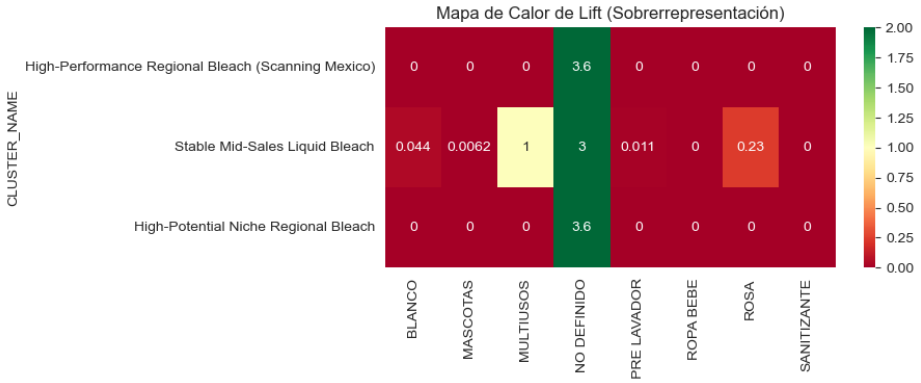
Análisis de atributos

```python
# Nombres de los clusters importantes a analizar
important_clusters=["High-Performance Regional Bleach (Scanning Mexico)","Stable Mid-Sales Liquid Bleach","High-Potential Niche Regional Bleach"]
# df_best = df[df['CLUSTER'].isin([2,3,7])] #Filtrando los datos con solo los clusters importantes a analizar

heatmap_lift(df=df, col_clusters='CLUSTER_NAME', categoria='ATTR1', figsize=(5,4), clusters_important=important_clusters)
heatmap_lift(df=df, col_clusters='CLUSTER_NAME', categoria='ATTR2', figsize=(5,4), clusters_important=important_clusters)
heatmap_lift(df=df, col_clusters='CLUSTER_NAME', categoria='ATTR3', figsize=(7,3), clusters_important=important_clusters)
```

✓ 1.3s Python





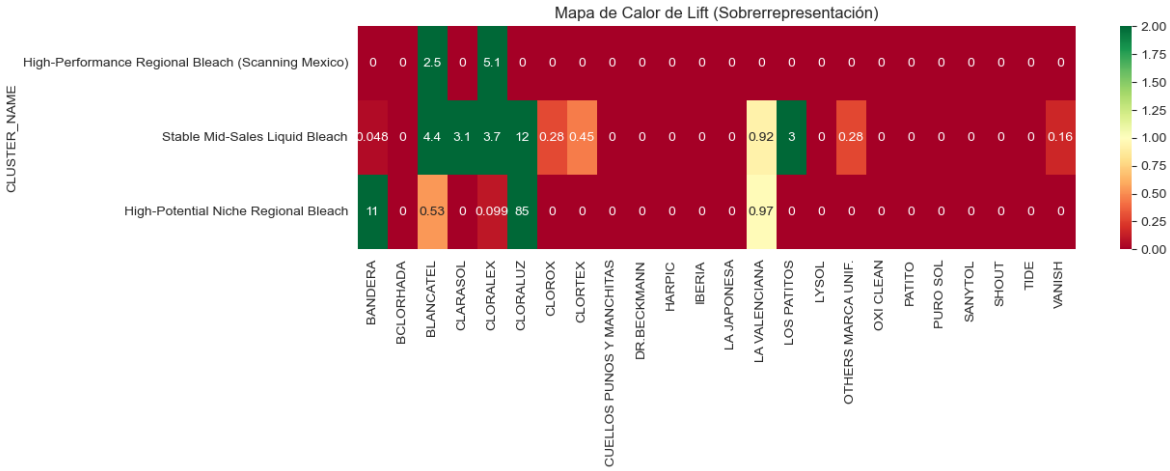


Podemos ver que en todos los clusters los atributos son "CLORO" y "NO DEFINIDO", lo cual no nos da más información de aquella que ya sabíamos.

Análisis de marcas

```
heatmap_lift(df=df, col_clusters='CLUSTER_NAME', categoria='BRAND', figsize=(12,3), clusters_important=important_clusters)
```

0.8s Python



```python
def brand_percent(df, cluster_name, brands): #Definiendo función para mostrar los porcentajes de las principales marcas que representan cada cluster
    print(f'- Cluster {cluster_name}')
    for brand in brands:
        df_cluster=df[df['CLUSTER_NAME']==cluster_name]
        n_cluster=len(df_cluster)
        n_brand=len(df_cluster[df_cluster['BRAND']==brand])
        percent=round(n_brand/n_cluster*100,1)

        print(f'El porcentaje de productos de la marca {brand} en el cluster es: {percent}%')
    print('')

brands1=['BLANCATEL','CLORALEX']
brands2=['BLANCATEL','CLARASOL','CLORALEX','CLORALUZ','LOS PATITOS','VANISH']
brands3=['BANDERA','CLORALUZ']

brand_percent(df=df, cluster_name='High-Performance Regional Bleach (Scanning Mexico)', brands=brands1)
brand_percent(df=df, cluster_name='Stable Mid-Sales Liquid Bleach', brands=brands2)
brand_percent(df=df, cluster_name='High-Potential Niche Regional Bleach', brands=brands3)
```

✓ 0.1s Python

```
- Cluster High-Performance Regional Bleach (Scanning Mexico)
El porcentaje de productos de la marca BLANCATEL en el cluster es: 4.3%
El porcentaje de productos de la marca CLORALEX en el cluster es: 95.7%

- Cluster Stable Mid-Sales Liquid Bleach
El porcentaje de productos de la marca BLANCATEL en el cluster es: 7.6%
El porcentaje de productos de la marca CLARASOL en el cluster es: 5.4%
El porcentaje de productos de la marca CLORALEX en el cluster es: 68.7%
El porcentaje de productos de la marca CLORALUZ en el cluster es: 1.5%
El porcentaje de productos de la marca LOS PATITOS en el cluster es: 5.9%
El porcentaje de productos de la marca VANISH en el cluster es: 3.7%

- Cluster High-Potential Niche Regional Bleach
El porcentaje de productos de la marca BANDERA en el cluster es: 85.2%
El porcentaje de productos de la marca CLORALUZ en el cluster es: 11.1%
```

Las marcas que tuvieron el mejor rendimiento (cluster "High-Performance Regional Bleach (Scanning Mexico)") fueron Blancatel y Cloralex, siendo esta última la que mayor presencia tiene en este grupo (representando un 95.7% de este cluster), por lo que esta marca es de las que puede llegar a ser de las más fuertes en el mercado actual.

En el caso del cluster "Stable Mid-Sales Liquid Bleach", Cloralex vuelve a aparecer, representando un 68.7% de estos datos. Aunque las demás marcas como Cloraluz, Blancatel, CLorasol y Los Patitos representan cada una menos del 10% de este grupo, lo cierto es que su lift es muy alta, indicando que una buena parte de estas marcas aparecen teniendo ventas medias y pronósticos históricos de ventas medios. Interesantemente, el 3.7% de este cluster lo conforman productos Vanish, pero tiene un lift muy bajo (de 0.16), por lo que son muy pocos las ventas de Vanish que realmente tienen estas ventas (lo cual se abordará más adelante),

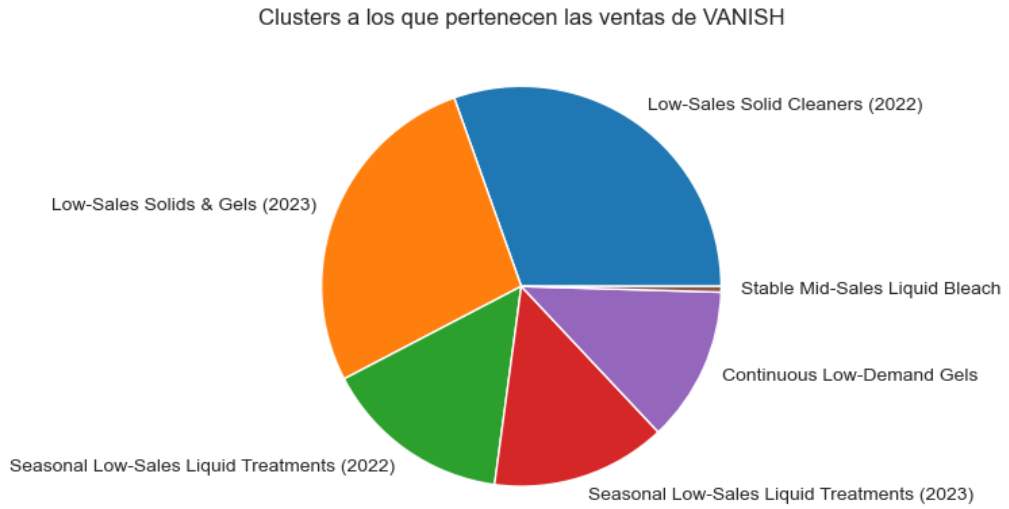
Por último, tenemos el cluster 'High-Potential Niche Regional Bleach', que está conformado principalmente por las marcas Bandera (85.2%) y Cloraluz (11.1%), las cuales han demostrado tener altos promedios de ventas históricas pero bajas ventas en este periodo.

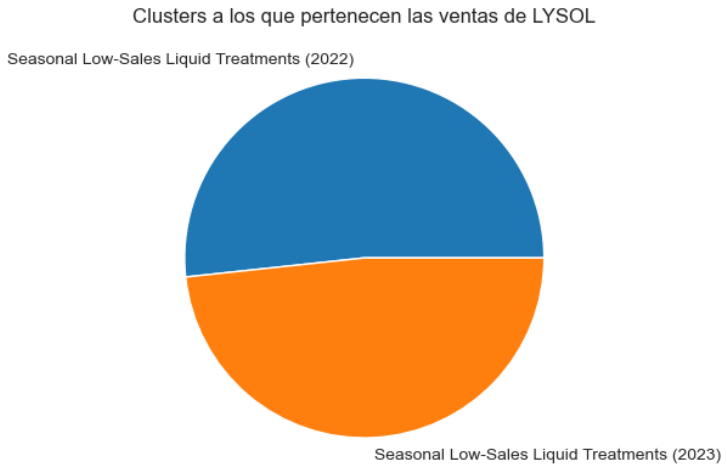
Marcas Vanish y Lysol

Para finalizar este análisis, es importante hacer énfasis en las marcas de interés: Vanish y Lysol. Para ello, filtraremos los resultados con solo estas marcas y visualizaremos la distribución y las características de las ventas agrupadas en los clusters encontrados por el modelo.

```python
df_vanish=df[df['BRAND']=='VANISH']
df_lysol=df[df['BRAND']=='LYSOL']

plt.pie(df_vanish['CLUSTER_NAME'].value_counts(), labels=df_vanish['CLUSTER_NAME'].value_counts().index)
plt.title('Clusters a los que pertenecen las ventas de VANISH')
plt.show()

plt.pie(df_lysol['CLUSTER_NAME'].value_counts(), labels=df_lysol['CLUSTER_NAME'].value_counts().index)
plt.title('Clusters a los que pertenecen las ventas de LYSOL')
plt.show()
```

✓ 0.8s Python




Clusters a los que pertenecen las ventas de LYSOL
Seasonal Low-Sales Liquid Treatments (2022)
Seasonal Low-Sales Liquid Treatments (2023)

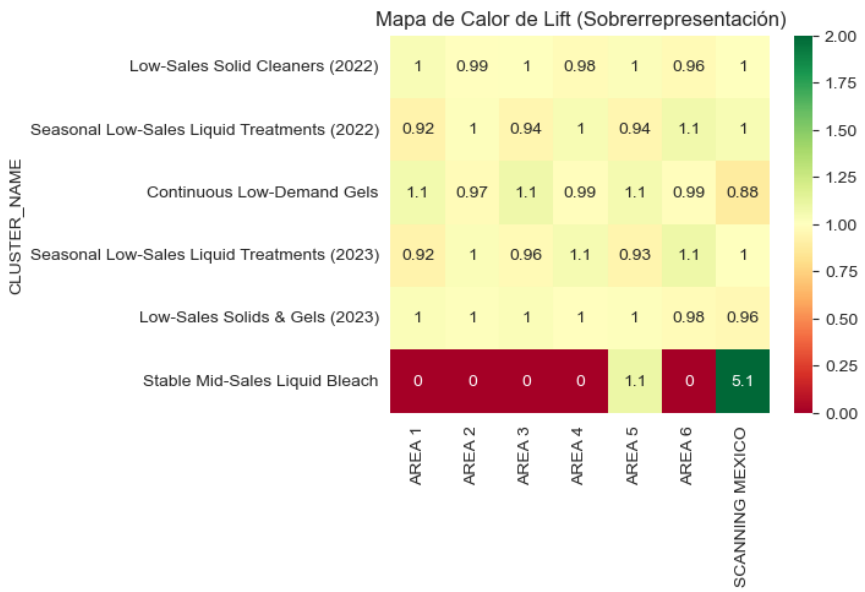
Podemos ver que la distribución de los clusters en la marca Vanish es muy parecida a la distribución general de todos los datos, salvo la ausencia del cluster "High-Performance Regional Bleach (Scanning Mexico)", lo que indica que Vanish no destacó por tener las mejores ventas en el periodo 2022-2023, pero sí llegó a tener una minoría de ventas con un rendimiento medio. Respecto a la marca Lysol, solo hay la presencia de dos clusters: "Seasonal Low-Sales Liquid Treatments (2022)" y "Seasonal Low-Sales Liquid Treatments (2022)", lo que indica que esta marca ha tenido un rendimiento bajo y no ha mostrado ser competencia para las otras marcas durante todo el periodo registrado.

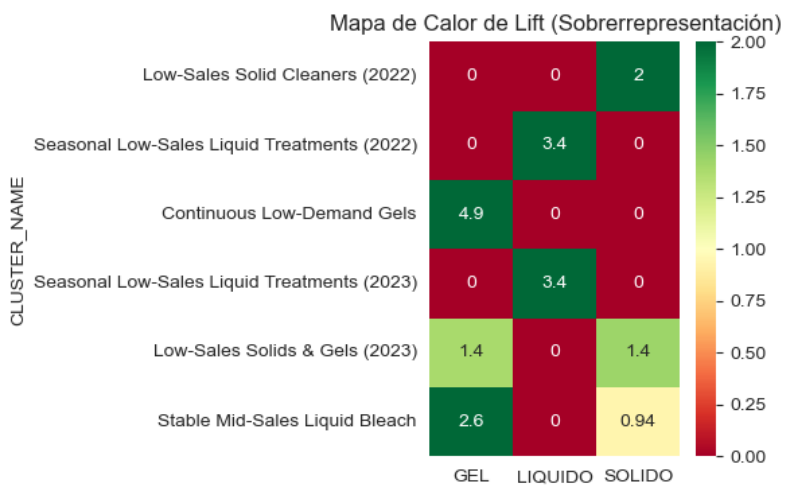
Ahora bien, se analizará la sobrerrepresentación (lift) de cada marca para las regiones, formatos y segmentos, para analizar si alguna clase de estas categorías destaca en los clusters que las componen

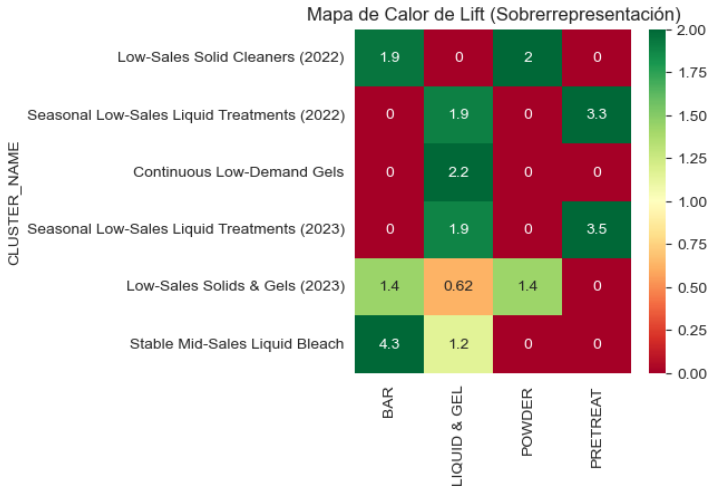
Vanish

```python
important_clusters=df_vanish['CLUSTER_NAME'].unique()

heatmap_lift(df=df_vanish, col_clusters='CLUSTER_NAME', categoria='REGION', figsize=(5,4), clusters_important=important_clusters)
heatmap_lift(df=df_vanish, col_clusters='CLUSTER_NAME', categoria='FORMAT', figsize=(3,4), clusters_important=important_clusters)
heatmap_lift(df=df_vanish, col_clusters='CLUSTER_NAME', categoria='SEGMENT', figsize=(4,4), clusters_important=important_clusters)
```

[62] ✓ 1.4s Python







Respecto a las regiones, resalta una sobrerrepresentación clara de la región Scanning Mexico en el cluster "Stable Mid-Sales Liquid Bleach", algo que resalta sis e compara con los datos de todas las marcas, pues en general, este cluster contenía una regiones mixtas, sin que destacara una particularmente. Esto sucede en el cluster que tiene el mejor rendimiento de Vanish, por lo que esta región es clave, no solo para cualquier marca, sino también para la marca Vanish particularmente.

Analizando los formatos, casi todos los clusters se comportan de la misma forma, a excepción del cluster antes mencionado. Para las ventas con el mejor rendimiento de Vanish, los productos de Gel son los que destacan, a diferencia de los productos Líquidos que destacan en el cluster para todas las marcas. Esto es importante de mencionar, porque esto indica que para Vanish, apostar por los productos de Gel existentes es también importante, además de apostar por más productos líquidos.

Por otro lado, lo que destaca y es diferente de la distribución total de los datos de ventas para la marca Vanish es, para el cluster:

- "Seasonal Low-Sales Liquid Treatments (2022)" destacan los productos del segmento 'Líquido y Gel'
- "Seasonal Low-Sales Liquid Treatments (2023)" destacan los productos del segmento 'Líquido y Gel'
- "Stable Mid-Sales Liquid Bleach" destacan los productos del segmento 'Barra'

Esto indica que los productos del segmento 'Líquido y Gel' les ha ido particularmente mal en todo el periodo 2022-2023. Además de que los productos de 'Barra' han destacado en sus mejores ventas, por lo que estos productos lograron ser competentes en algunos momentos de 2022 y 2023.

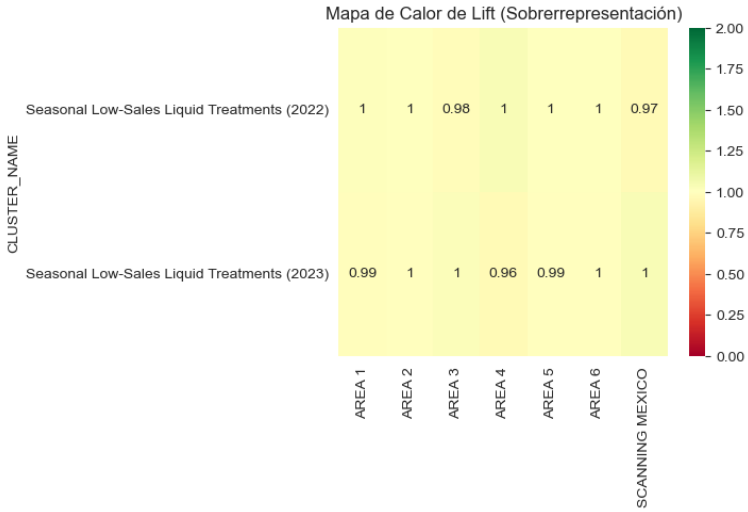
Lysol

Generate | Code | Markdown

Add Code Cell

```python
important_clusters=df_lysol['CLUSTER_NAME'].unique()

heatmap_lift(df=df_lysol, col_clusters='CLUSTER_NAME', categoria='REGION', figsize=(5,4), clusters_important=important_clusters)
```

[64] ✓ 0.4s Python



Podemos ver que en ninguno de los clusters de las ventas de Lysol destaca alguna región, indicando que las regiones de sus ventas son mixtas como las ventas totales.

Nota: No se graficaron los mapas de calor para las clases de Formato y Segmento, porque estas solo están conformadas por una clase cada una

```python
# Comprobando que
print(df_lysol['FORMAT'].unique())
print(df_lysol['SEGMENT'].unique())
```

[44] ✓ 0.0s Python

```
['LIQUIDO']
['SANITIZER']
```

Por lo tanto, solo se puede concluir para Lysol que la venta de sus productos liquidos sanitizantes, los cuales representan toda su línea de productos, tuvo un mal desempeño en todas las regiones en las que fueron vendidos.

Guardar los resultados y el modelo

```python
# Guardado de datos ya agrupados en clusters
df.to_csv('Final_DataFrame_clustered_FINAL.csv')
```

[45] ✓ 2.3s Python